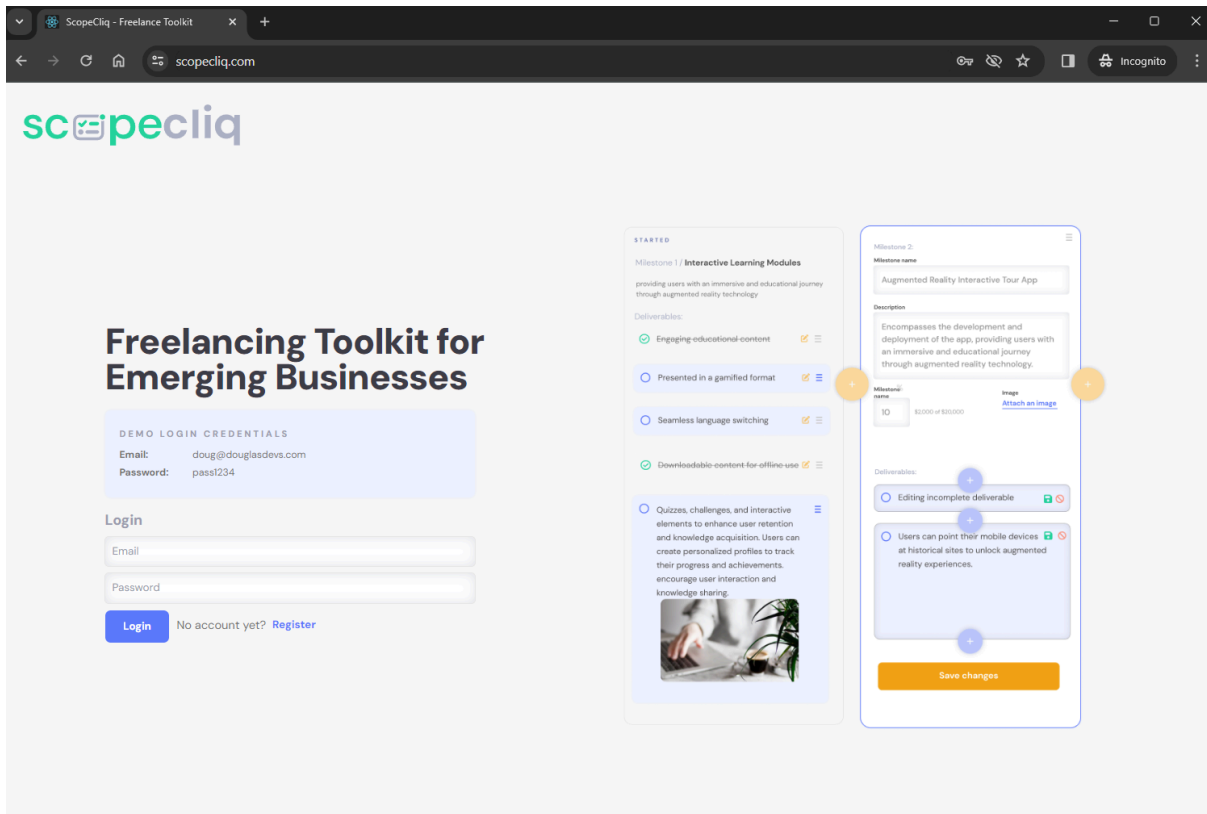




Freelance Toolkit

Margaret (Macci) Macaranas

December 2023



<http://www.scopecliq.com>

Table Of Content

Table Of Content	3
I. Introduction	7
1. Background	7
2. Problem Statement	7
3. References and Market Research	7
5. Assumptions	8
II. Proposed Project	9
1. ScopeCliq	9
2. Why does the market require ScopeCliq?	9
3. Benefits	10
4. Scope and Limitations	10
III. Project Blueprint Feature	11
1. Projects	11
What is a Project?	11
Projects Cards	11
Project Status	12
Creating a new Project	12
Project Domain	12
2. Project Blueprints	13
What is a Project Blueprint?	13
3. Milestones	14
What is a Milestone?	14
Milestone Status	14
Scroll To Ongoing	15
Milestone and Invoices	15
Creating, Updating, or Deleting a Milestone	16
4. Deliverables	17
What is a Deliverable?	17
Creating and Updating a Deliverable and Status	17
IV. Client Portal and Consultant Dashboard Feature / Views	18
What is the difference between a Client Portal and Consultant Dashboard?	18
1. Client Portal	18
How can I find a Project's Client Portal link?	19
Portal Login / Password Protection	19
Notification Sidebar	20
Project Details	20
2. Consultant Dashboard	21
What is the Consultant Dashboard?	21
2.1. Consultant Dashboard > Consultant Project Page	21
What is the Project Page?	21

Accessing the Project Page	22
(1) You can access the Project Page by clicking on a “Go to project” button from a Project in the Project List at Consultant Homepage (2) Or access a Project Page with this URL format <a href="http://scopecliq.com/domain/<project_id>/">http://scopecliq.com/domain/<project_id>/ .	22
Actions in the Project Blueprint in Project Page	22
Notification Sidebar	23
Project Details	23
2.2 Consultant Dashboard > Consultant Homepage	24
What is the Consultant Homepage?	24
How to access or navigate to Consultant Homepage?	25
Project List	25
Adding A New Project	26
Your Organization Details	26
Summaries and Analytics	27
V. Other Features and Functionalities	28
1. Notifications	28
What is a Notification?	28
Types of Notification and Trigger Actions	28
Difference of Notification in Client Portal vs Consultant Dashboard?	28
2. Invoices	29
What is an Invoice?	29
Global Invoice Link for a Single Invoice Item and Accessibility	30
Invoice Lists Entry Points and Accessibility	30
Invoices and Invoice Lists on Client Portal versus Consultant Dashboard	31
Client Portal Invoices	31
Consultant Dashboard Invoices	31
Generating an Invoice	32
Payments and Stripe Integration	32
3. User Management	33
Consultant versus Client	33
The Landing Page	33
Consultant / The User Object	33
Login	34
Organizations	34
Registration Modal	34
Updating your Account	35
Uploading a Logo or Avatar	35
VI. Learnings	36
1. Learnings	36
2. Next Phases	36
VII. Appendix	37
1. Installation Guide	37
System Requirements and Technology Stack	37

How to Run this Project	37
Troubleshooting	38
2. User Guide	39
Login and registration	39
Adding your first project	39
Adding your first milestone and deliverables	39
Adding your first invoice	39
Clients to pay your invoice	39
Updating your profile	39
3. Architecture	40
Technology Stack	40
Deployment	40
Siteground	40
Using npm run build	40
Using PuTTY and SSH	40
4. Backend Implementation Codebase	41
4.1. Database Design and Relationships	41
Design	41
Relationships	42
User – Organization – Project:	42
Project – Milestone – Deliverables:	42
Project – Milestone – Invoice:	42
Notifications – Deliverables – Milestones – Invoices – Projects:	42
4.2. Tables and Migrations	43
Migrations	43
4.3. Seeders	45
4.4. Routes and APIs	46
api.php	46
4.5. Controllers	48
User Controller	48
Organization Controller	48
Project Controller	49
Milestone Controller	50
Deliverables Controller	51
Invoices Controller	52
Notification Controller	53
Analytics Controller	55
5. Frontend Implementation Codebase	58
5.1. Routes and Views	58
App.js	58
Index.js and config.js	59
Styling	60
style.scss	60

5.2. Components and Modules	61
5.3. React-redux or the Store	62
5.4. Notable Features and Snippets	63
Snackbars	63
Use of the browser's local storage	64
React Material UI Design Library	65
Stripe	66
Invoice.js	66
References	67

I. Introduction

1. Background

- The market for freelancers worldwide with a rate of growth of 15%.
- 47% of working millennials are freelancers or approximately 1.57 billion freelancers globally.
- By 2027, the worldwide marketplace for freelance platforms will be more than \$9.19 billion.

*Freelancing Statistics – By Wages, Demographic, Market Share, Earning, Trends ¹
Are We Ready For A Workforce That is 50% Freelance? ²*

2. Problem Statement

As freelancers and small businesses, you are specifically hired for your specialised skills but every project requires a big chunk of account management: administration, customer care, budgeting.

These unaccounted responsibilities often consume valuable energy that could otherwise be directed towards core tasks.

Many excel in their field but struggle with organisation skills and/or client communication. The demands in time and energy allocated for account management is frequently underestimated and not always included in the project brief or fees.

3. References and Market Research

To come up with this solution, we reviewed several software and their usages.

There are properties of existing account management systems + task management systems that we are confident can work together if simplified. We took the most used features and applied into a freelance system that focuses on what gives clients a clear overview of how the project is progressing.

Popular Account Management Systems (targeted to growing businesses)

ScopeCliq differs from other Account Management systems because we focus on implementing the project to completion.

- Dubsado³ – Focuses more on finding, selling to, and onboarding clients

- Indy⁴ – Focuses on tasks within the team. It is more internal facing than client facing
- HoneyBook⁵ – Focuses more on invoicing

Task Management Systems

ScopeCliq stands apart from other task management systems due to its unique focus. Unlike many task management tools that predominantly centre their attention on internal team dynamics, ScopeCliq's distinctive approach is client-oriented.

Items in popular task management softwares typically contain abundant details that are not essential to the client's point of view and may just confuse or intimidate a client.

- Jira⁶
- Asana⁷
- Trello⁸

5. Assumptions

Projects follow the similar **Project Blueprint – Milestone – Deliverable** model. The vocabulary might vary (e.g. *Project Plan – Phases – Tasks* or *Feature – Sprint – Ticket*) but the concepts and categorization are similar

Another assumption is most **big and medium sized businesses start from small teams / businesses** similar to freelancers, independent contractors, and gig workers

.

II. Proposed Project

1. ScopeCliq

ScopeCliq is a straightforward freelancing toolkit that helps you focus on your core tasks by reducing client update requests, improving client alignment, and supporting project + project management.

With ScopeCliq you get the following:

Clear Source of Truth

Client and Consultant have one clear source of truth which is the Project Blueprint holding all the Deliverables grouped by Milestones providing a central reference point

Dedicated Client Portal

Consultant can send a professional Client Portal link where Client can actively track the project's real-time status through the Project Blueprint.

Consultant Dashboard

Consultant will have a Consultant Dashboard where they can track and manage clients and projects with an overview of where they are as a business.

2. Why does the market require ScopeCliq?

ScopeCliq distinguishes itself from other available systems because of our three primary focus:

- 1.) We address a common overlooked for weak point: prioritising client alignment and updates using the Client Portal
- 2.) We focus beyond account management, the Project Blueprint aim to guide implementation of project to completion,
- 3.) We invested a lot of planning in our user experience to ensure simplicity and ease-of-use to ensure accommodation of all kinds of business industries not limited to technology, design, and marketing

We provide all of the above while still providing the basics such as account management, task management, invoice management, notifications, and analytics.

3. Benefits

We are assuming the following benefits:

Professional looking updates

The consultant will maintain a dynamic and professional page to showcase their work to the client.

Reduced update requests

This will minimize the need for clients to frequently request updates during the consultant's task-focused periods.

Satisfaction from completed

Clients will find satisfaction in seeing deliverables being completed and seeing the project move forward. There is also a positive experience for Consultants to tick off items in the project plan.

Seamless and more transparent communications

The platform will serve as a transparent space for consultants and clients to align and gain a clear understanding of the project's status and scope. Aside from the Project Blueprint offering progress, the use of Notifications allows Clients to acknowledge the activities completed.

Easy invoicing and payments

The Invoicing and payment process is simplified. Every milestone is assigned a monetary value which the Consultant can invoice once completed. The invoice link can be easily sent to Client which can then be easily paid with our Stripe integration.

4. Scope and Limitations

What ScopeCliq is not

ScopeCliq is not intended to replace Consultant's customer support services for Clients. Instead, its primary purpose is to provide a transparent space where Consultant and Client can align and have a clear understanding of the project's status and scope.

III. Project Blueprint Feature

The screenshot displays a project card for 'Siesta Farm and Restaurant'. At the top, the project name is in large white text, followed by a status indicator 'ONGOING' in blue. A circular progress bar on the right shows '50%' completion. Below this, a table lists project details: Brief (Marketing package. Farm to table artisanal easy dining), Due (No due date), and Total budget (\$40,600.00). The client information section, 'Julia Studios', lists Name (Julia Evers, Julia Studios est 1992), Email (director@juliastudios.com), and Phone (+63 234 567 8901). A 'Dedicated Portal' section provides a URL (http://localhost:3000/portal/siesta) and a password (pizza) with a 'Go to portal' button. The 'Terms and Conditions' section contains three paragraphs of text regarding content, images, and revisions.

Siesta Farm and Restaurant	
ONGOING	
50%	
Brief	Marketing package. Farm to table artisanal easy dining
Due	No due date
Total budget	\$40,600.00

Julia Studios	
Name	Julia Evers, Julia Studios est 1992
Email	director@juliastudios.com
Phone	+63 234 567 8901

Dedicated Portal

URL: http://localhost:3000/portal/siesta

Password:

[Go to portal](#)

Terms and Conditions

Text Content – All content material (copy, product images and details, etc) is to be provided by the client. Should more content be needed, we can arrange for Content Creation to complete missing pieces of the website. (*See add-ons)

Images – We primarily use images provided by clients. Additionally, we are free to use CC0 License images (a.k.a. Free License Images) available online.

Requests and revisions – Realistically, revisions are always part of any design process which is why revision rounds are allotted to the project. Once designs are approved and implemented on the

1. Projects

What is a Project?

You can define a Project with a title, brief description, due date, total budget, and terms & conditions.

A project will be owned by a Client Organization. An **Organization** object has an organization name, address, contact person, email, and phone.

A project will also be provided a **Client Portal**.

A project will also have its own **Invoice List**.

And ofcourse a project will have its own comprehensive **Project Blueprint**.

As the Project Blueprint progresses, the project completion status is dynamically updating

Projects Cards

A project is visually represented using **Project Cards** in three forms: (1) **Collapsed** form, (2) **Expanded** form, (3) **Full** form.

Figure 4.1.1. Project Card in Full Form. You can find the project status and completion rate on the upper right corner.

PENDING (NOT STARTED) 0%

Lettered Leather

Brief: Marketing package. Local leathered goods
 Due: Dec 25, 2023
 Total budget: \$8,000.00

Stationery Company

Name: Angela Sharma, Accounts Manager
 Email: angela@office.com
 Phone: +1 234 567 8901

Collapse ^ View invoices Go to project

Figure 4.1.2. (left) Project Card, Collapsed Form
 Figure 4.1.3 (right) Project Card, Expanded Form

COMPLETE 100%

Truck Flow Digital

Brief: Logistic system
 Due: Sep 01, 2023
 Total budget: \$4,200.00
 Client: Julia Evers, Julia Studios

Show more ^ View invoices Go to project

Project Status

The **Project Status** updates dynamically as we progress through the project.

There are three statuses for a Project: (1) **Pending / Not Started**, (2) **Ongoing**, (3) **Complete**.

Pending / Not Started simply means no deliverable has been completed for a project. Once a single Deliverable has been completed, the whole project is marked **Ongoing**. Once all **Deliverables** are completed, the project is marked **Complete**.

The Project Completion Rate is available in a circular progress at the upper right of the Project Cards.

KPI Summaries Dive in existing projects Start a new project

Projects

Client

CLIENT'S ORGANIZATION

Stationery Company

Project

PROJECT DETAILS

General Information

Project Name

About the Project

\$ Project Budget

Target Completion Date (Optional)

yyyy-mm-dd

Portal information

http://localhost:3000/portal/ Portal Domain Portal Password

Terms

Terms and Conditions

Add

Creating a new Project

You can create a new project from the Consultant Dashboard 'Start a new project'.

You can choose one of your existing Organizations as the client of Project create a new Organization.

Figure 4.1.4 Create a new Project modal

Project Domain

From creating a new project you can already assign the Client Portal *link* or what we call **Project Domain** and a *password* that you can provide the Client. Find out more: [How can I find a Project's Client Portal link?](#)

2. Project Blueprints

The **Project Blueprint** is the foundation of ScopeCliq. It is what allows the Client and Consultant to have a centralized reference of the project scope and project progress.

What is a Project Blueprint?

Every **Project** has one **Project Blueprint** which is the agreed upon project plan that guides the Project implementation to completion. This contains the **Milestone** components which houses its own group of **Deliverable** components.

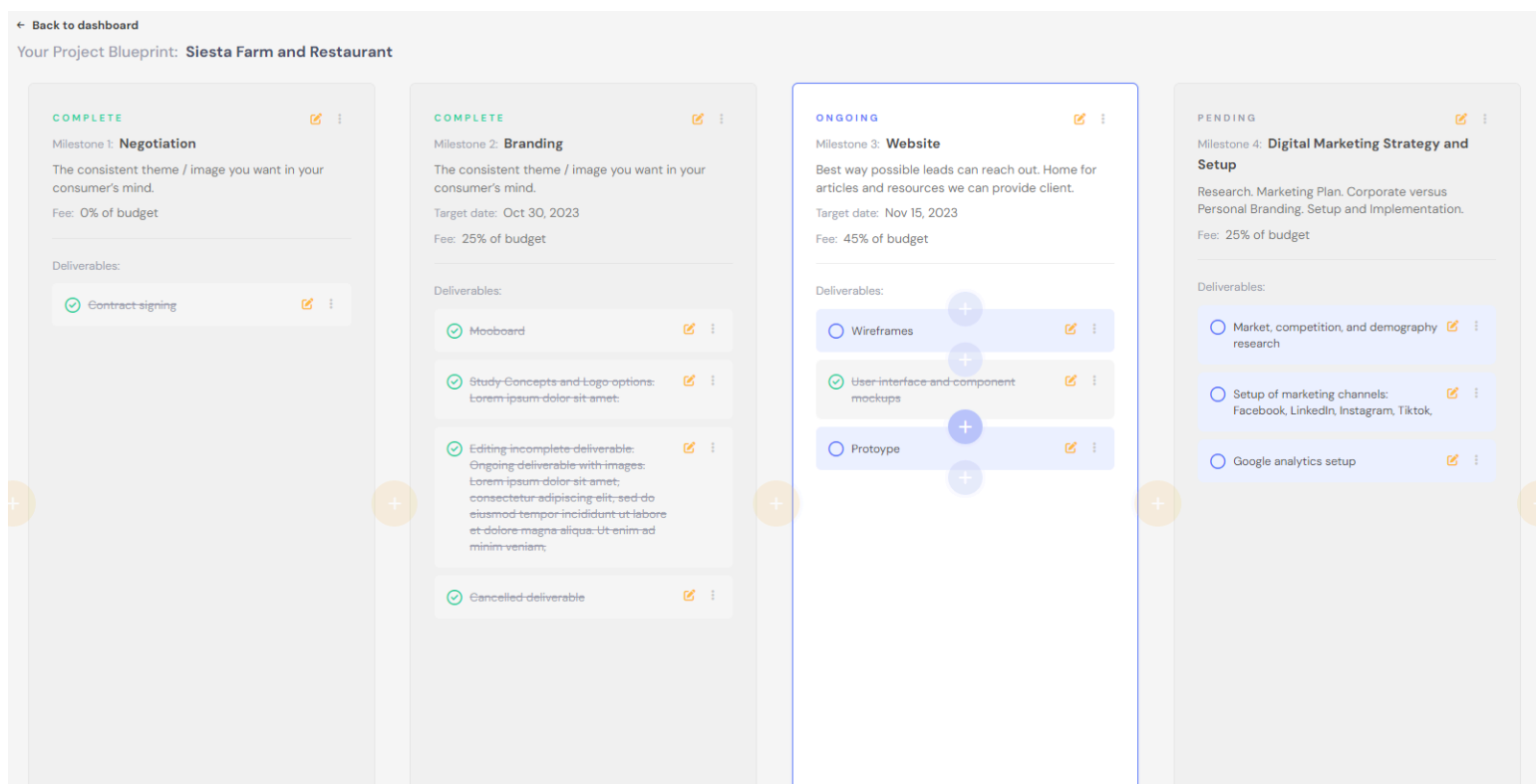
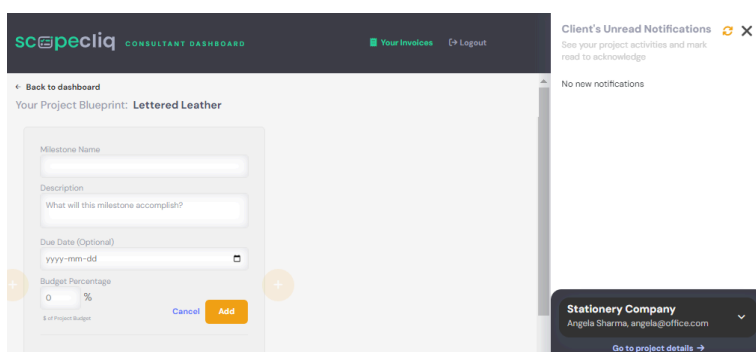


Figure 4.2.1. An example of the Project Blueprint in Edit Mode



The Project Blueprint has two modes: **View Only Mode** seen on the Client Portal and the **Edit Mode** seen on the Consultant Dashboard.

Figure 4.21 If the Project Blueprint is empty and in Edit Mode, an empty editable Milestone is immediately visible to encourage the Consultant to create his first Milestone.

3. Milestones

What is a Milestone?

Milestones refers to a specific achievement required for the project to progress.

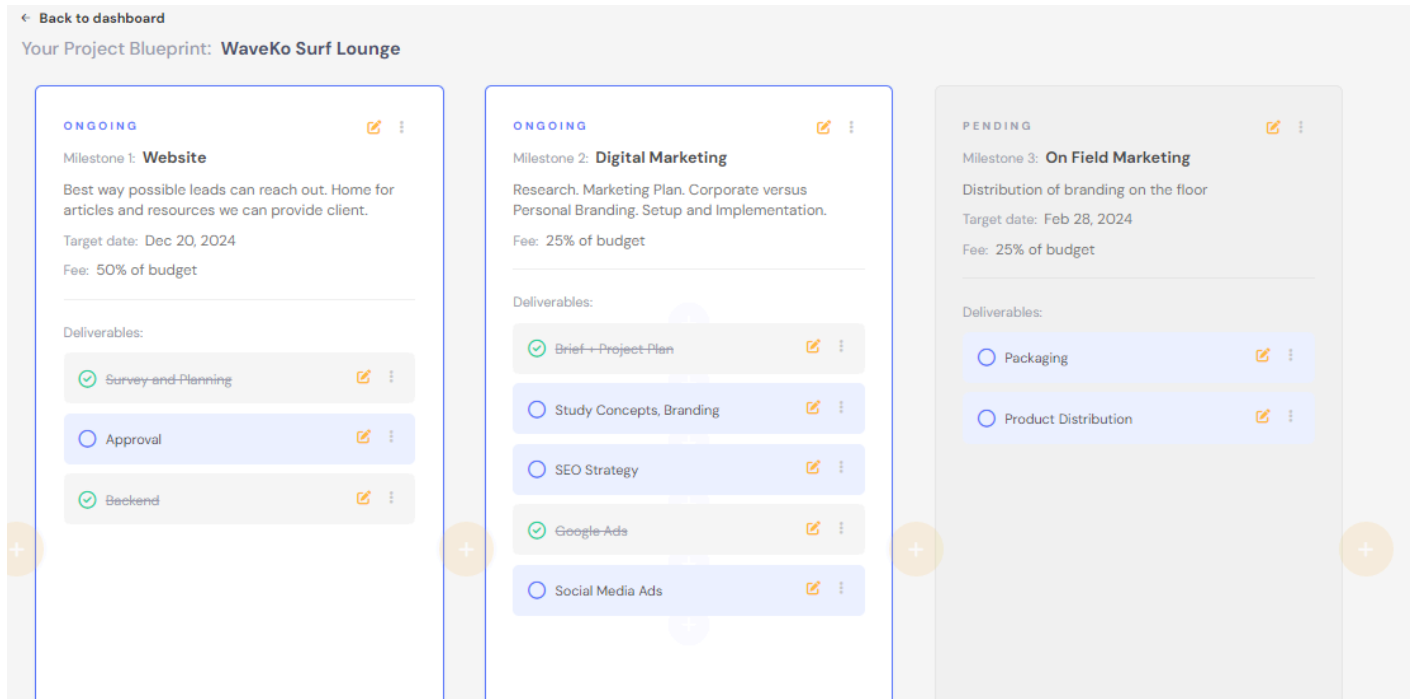
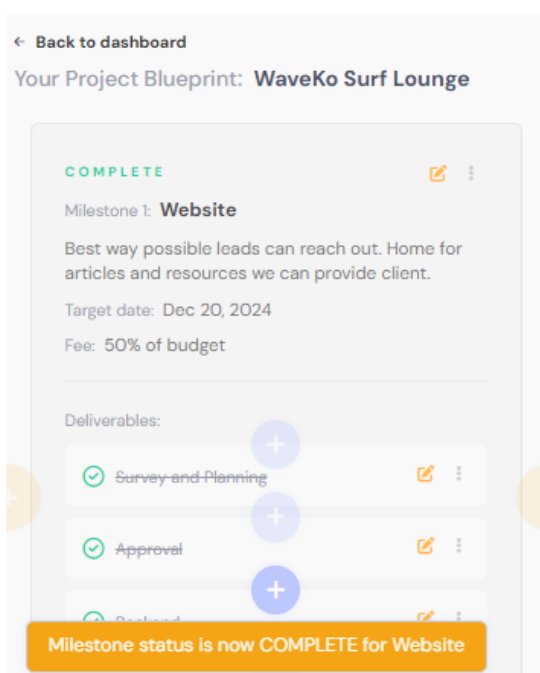


Figure 4.3.1.. Four states of a Milestone in reverse order: Complete, Ongoing, Pending, Not yet created / Edit Mode. You can also see the UI differences depending on the milestone status.

A milestone has a *name*, *description*, *due date* (optional) and a *budget percentage*. The **due date** is optional but will allow for analytics for when the milestone is due. The **budget percentage** defines the value of a milestone in reference to the project.



Milestone Status

Same as Project, there are three statuses for a milestone: (1) **Pending**, (2) **Ongoing**, (3) **Complete**. The **Milestone Status** updates dynamically as you go through the deliverables of the milestone.

Figure 4.31. On the bottom left the notification that the status has been updated from Ongoing to Complete is displayed

Pending / Not Started simply means no deliverable is completed for the milestone. Once a deliverable is

completed, the milestone is marked **Ongoing**. When all deliverables are completed, the project is marked **Complete**.

From *Figure 4.6*. You can see that three statuses are color coordinated in the following manner:

Pending – Very gray and muted so you do not focus on it

Ongoing – Highlighted with popping white background, shadow, and bright purple outline to imply that this should be the consultant's current focus

Complete – Green to emit a positive sense of success for both Client and Consultant

Scroll To Ongoing

Once all milestones fetched and statuses are displayed, **the browser scrolls to the first Ongoing Milestone** to imply that this is what the consultant should focus on completing.

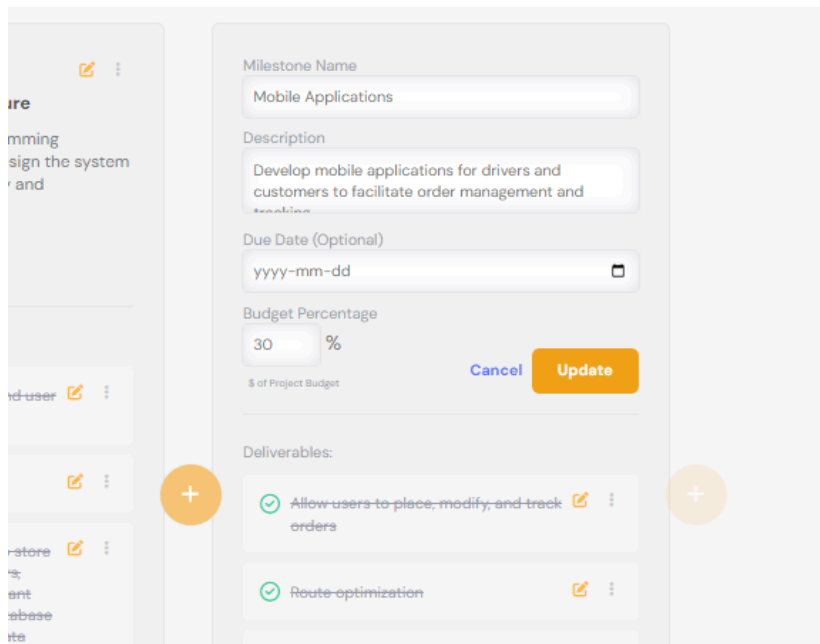
Milestone and Invoices

The *budget percentage* set when creating or updating a milestone dictates the receivable amount on the milestone's respective invoice.

Figure 4.3.2.. On this invoice snippet you can find the total receivable amount of an invoice respective to the milestone in relation to the Project Budget

Milestone 1: Website	
Invoice #003	Total: \$ 44,000.00
	Fee: 50% of \$ 88,000.00

Creating, Updating, or Deleting a Milestone



The screenshot shows a form for editing a milestone. The form is titled 'Edit Milestone' and contains the following fields:

- Milestone Name:** A text input field containing 'Mobile Applications'.
- Description:** A text area containing 'Develop mobile applications for drivers and customers to facilitate order management and tracking'.
- Due Date (Optional):** A date picker field showing 'yyyy-mm-dd'.
- Budget Percentage:** A numeric input field showing '30' followed by a '%' symbol.
- Buttons:** 'Cancel' (blue) and 'Update' (orange) buttons.
- Deliverables:** A list of deliverables with checkboxes and edit icons. The first deliverable is 'Allow users to place, modify, and track orders' (checked). The second is 'Route optimization' (checked).

Adding a milestone. A golden Add button is placed at both side of existing milestones for you to decide where you want to place your new milestone.

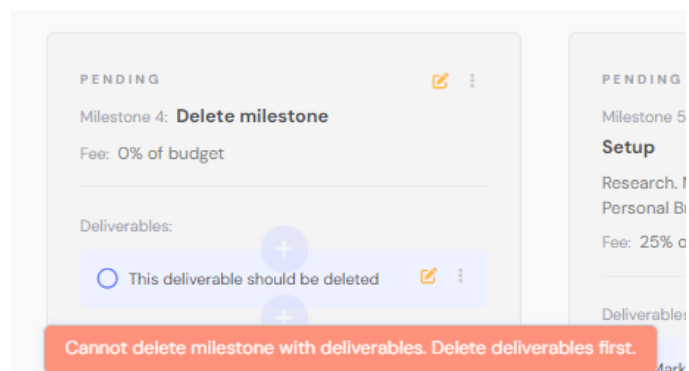
After saving the milestone will be saved in correct position.

Figure 4.3.3. This is an example of an existing milestone in edit mode.

Editing a milestone. You can click the edit icon on the upper right corner or simply click on the icon to show the pre-filled Edit Milestone form.

Deleting a Milestone. The option to delete a milestone is inside the menu on the upper right. A milestone is only deletable if it has no associated Deliverable.

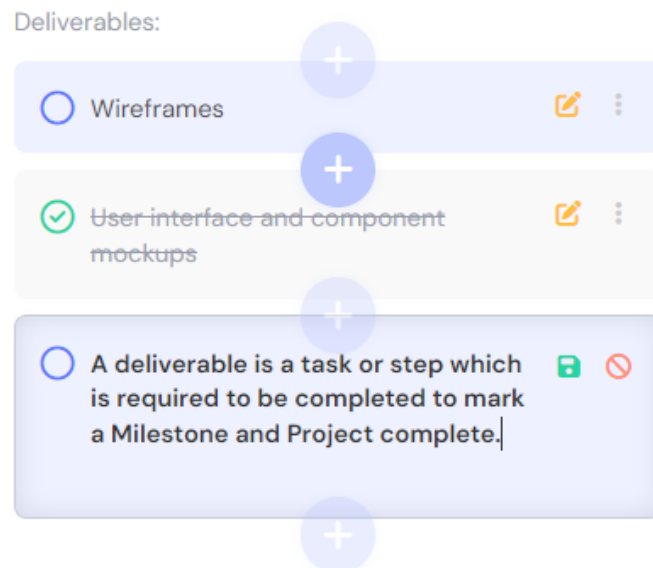
Figure 4.3.4 Attempting to delete a milestone



4. Deliverables

What is a Deliverable?

A deliverable is a task or step which is required to be completed to mark a Milestone and Project complete.



In ScopeCliq it is represented with the tickbox and description.

Figure 4.3.1. A set of editable deliverable

We chose the word deliverable because the outcome may or may not be intangible. It can be in the form of a task, document, or even just a verification. Importantly it can be verified with **Complete** or **Incomplete**.

Creating and Updating a Deliverable and Status

Toggling Deliverable Status. Simply click the round toggle icon on the left to mark as *Complete* or *Incomplete*. If completed it will have a checkmark and remain empty if incomplete.

Editing a Deliverable. Similar to Milestones clicking on the description will display the form to edit deliverable description. The whole component will seem like one field where you write the description. The component will also automatically match the length of your description. The save and cancel icons are found on the upper right in place of the edit and menu icon.

Adding a Deliverable. Similar to Milestones clicking add buttons are available before and after existing deliverables to enable adding a new deliverable. These buttons are only visible when you hover to reduce visual clutter. Once clicked a new empty deliverable component will take the place of the button. After saving, the deliverable is placed in the correct position.

IV. Client Portal and Consultant Dashboard Dashboard Feature / Views

What is the difference between a Client Portal and Consultant Dashboard?

Both are views for the **Project Blueprint** plus with a sidebar for supporting features and information.

Simply a Client Portal is for the Client and is not editable. A Consultant Dashboard is for the Consultant where you can build and update the Project Blueprint.

In the Client Portal, you can mark notifications as read, while in the Consultant Dashboard, you cannot.

Another easy way to visually distinguish them is the Navigation is light for the Client Portal and dark for the Consultant Dashboard.

Both Client Portal and Consultant Dashboard have some other subpages, such that they both have an Invoice List, but only Client Portal has a **Portal Login** while Consultant Dashboard have **Consultant Home Page** and **Consultant Project Page**.

Another important differentiation is that a Client Portal is respective to one Project, while a Consultant Dashboard houses multiple Projects.

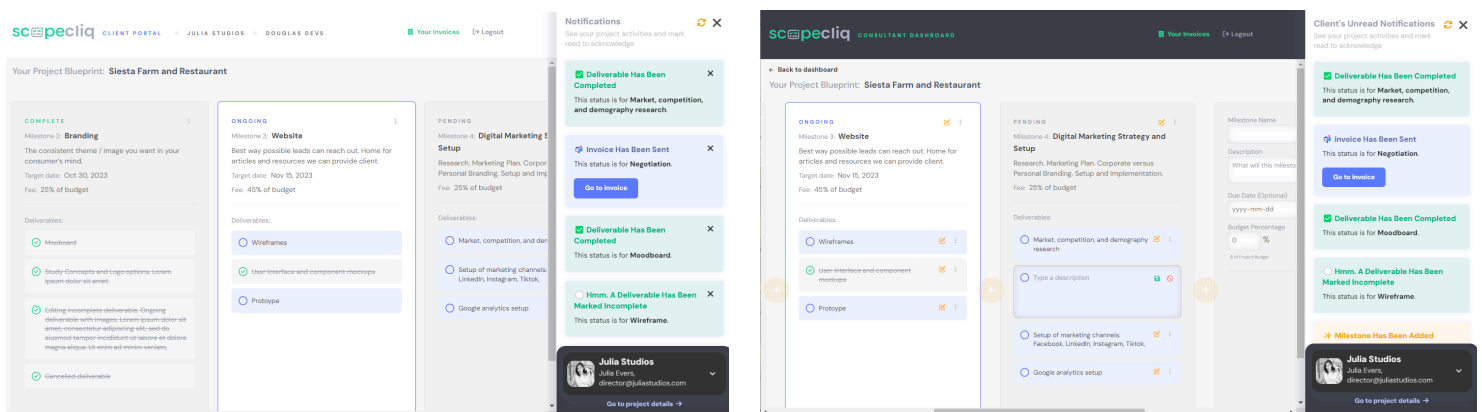


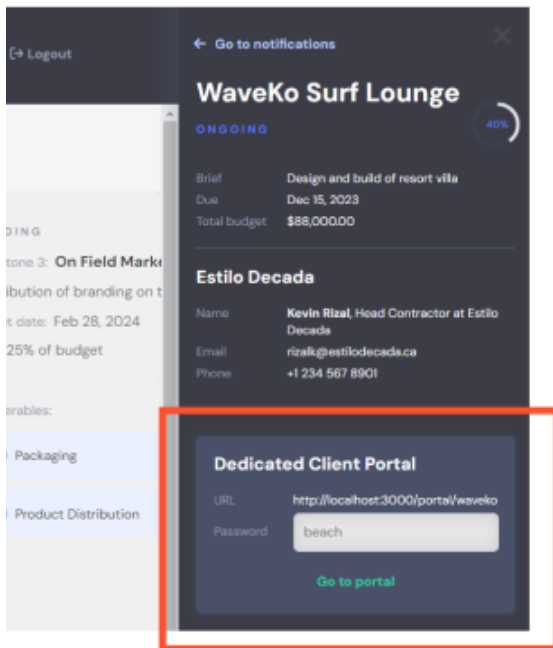
Figure 5.0 (left) Client Portal view ; Figure 4.2.02 (right) Consultant Dashboard view of same Project

1. Client Portal

When creating a project, the Consultant can assign a dedicated Client Portal link for the Client to conveniently project progress and activities.

This password-protected web page serves as the go-to url that Clients can review anytime they need the project status. The Client Portal reduces the need for frequent update requests and improves alignment between Clients and Consultants.

The convenience is that Clients are not required to sign up to access a project's Client Portal, but it is protected by a password that the Consultant will provide.



How can I find a Project's Client Portal link?

You can find the Client Portal's link from the Sidebar. Select the "Go to project" button at the bottom under the Client's Information.

You will be provided with the link and the respective password.

A **Portal Domain** is a **unique keyword** you assign to your project to be used for the Client Portal Link.

The Client Portal link is normally the format below:
<http://scopecliq.com/portal/<project-domain>>

Figure 5.1.1. Find the Client Portal information on the Project Details section of the sidebar

Portal Login / Password Protection

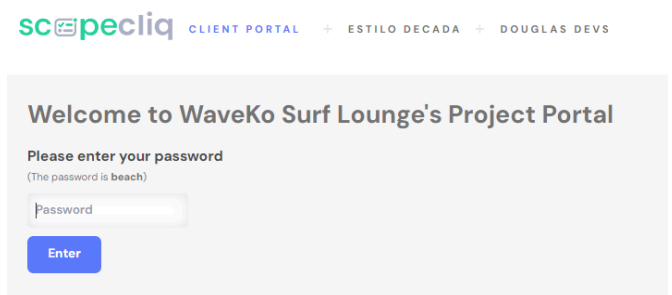
To entry point of the Client Portal is the **Portal Login** where you are required to enter the correct password to be provided also by the Consultant.

(For demo purposes, you can explicitly find the password on the same page.)

Figure 5.1.2 Logging In the Client Portal

From the Portal Login you can already see the navigation with the Client Organization Name and Consultant Organization Name.

Once credentials are entered, user will be navigated to the Client Portal with Project Blueprint.



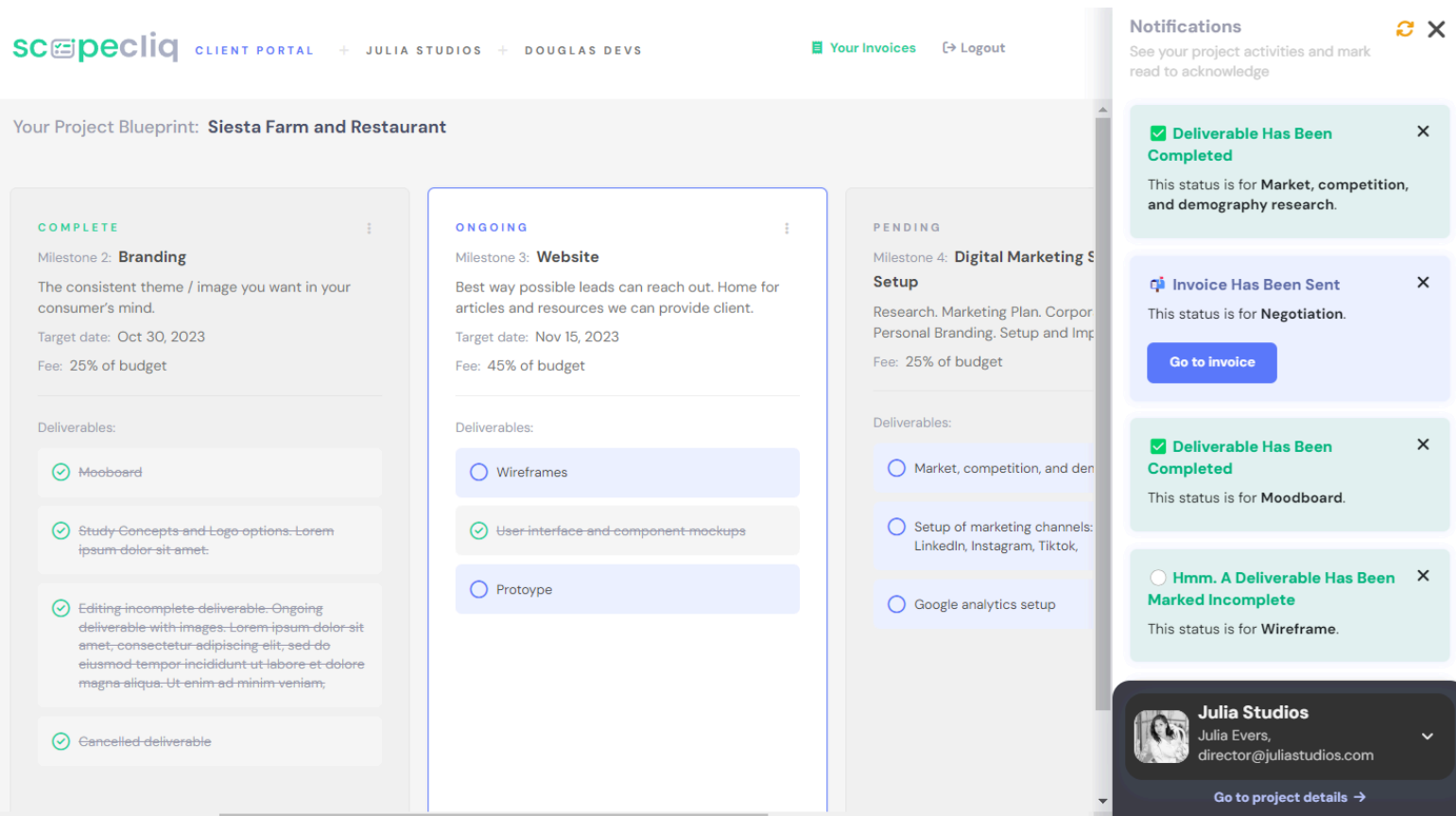


Figure 5.1.3. Closer view of Client Portal

Notification Sidebar

Notifications

See your project activities and mark read to acknowledge



✓ **Milestone Has Been Completed**

This status is for **SEO Strategy**.



Mark read

✓ **Deliverable Has Been Completed**

This status is for **Social Media Ads**.



A **Notification** is basically a log of the activities in the project.

The activities are shown on the sidebar so the Client will be notified of the latest changes / progress.

The Client will then be able to **mark the notification as read** by clicking on the X button on the upper right.

Figure 5.1.4. A Notification and the tooltip that acknowledges closing the notification marks it as read

Project Details

The details of the project area available on the sidebar as well as discuss on section: [What is a Project?](#)

2. Consultant Dashboard

What is the Consultant Dashboard?

The Consultant Dashboard is the view of the Consultant. It has two major modules: (1) the Consultant Dashboard > **Consultant Project Page** where we build and update the projects and (2) the Consultant Dashboard > **Home Page** where we find all the Projects and see the health of the Consultant's business through analytics / summaries.

As mentioned above, all the pages that are categorized as Consultant Dashboard have a dark navigation that explicitly states Consultant Dashboard.

2.1. Consultant Dashboard > Consultant Project Page

What is the Project Page?

We finally go full circle with the Project Page. In the Project Page we see a similar view as the Client Portal, but the Project Blueprint is now in **Edit Mode**. In this page we are able to accomplish all the actions that were mentioned on the Project Blueprint section.

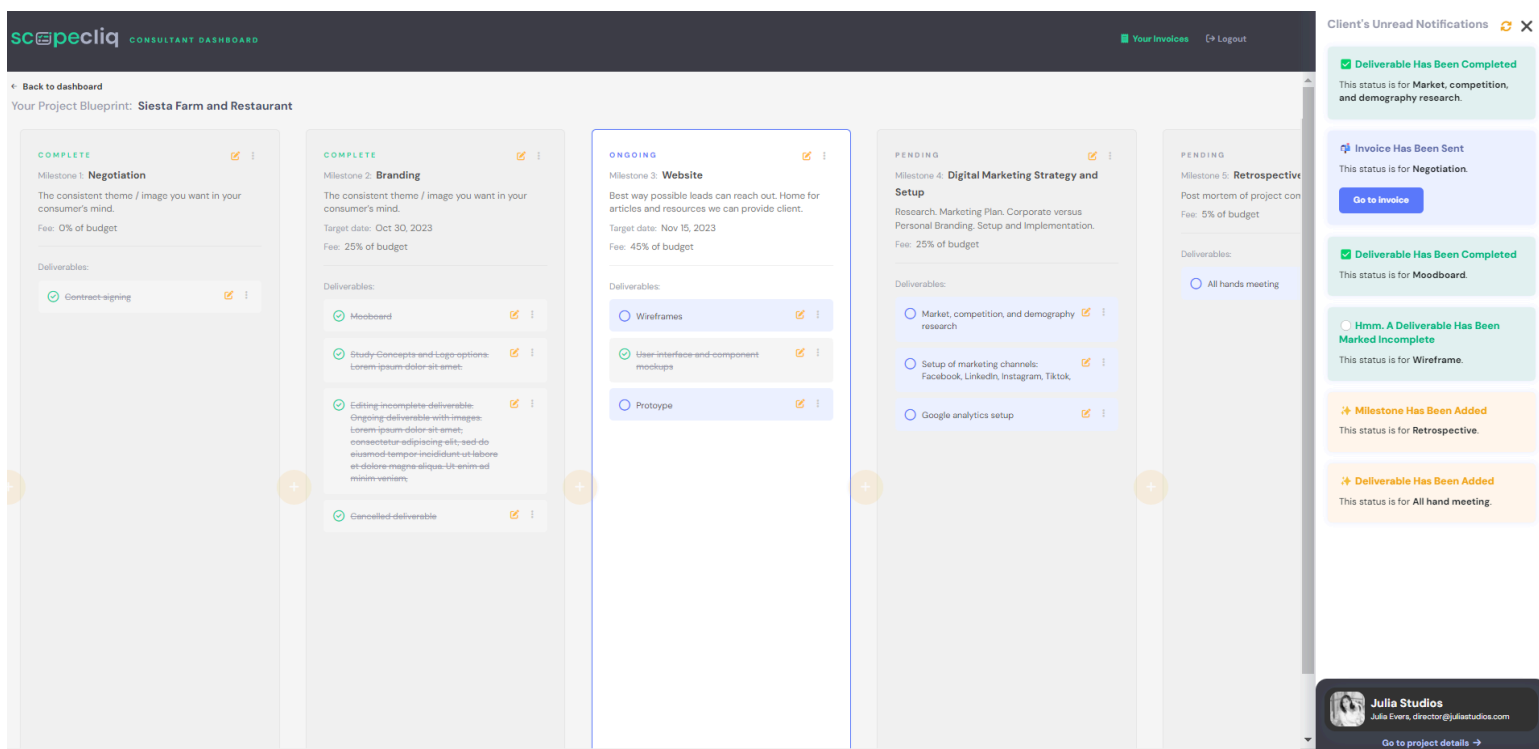


Figure 5.2.1.1. Closer look at the Consultant Project Page

Accessing the Project Page

(1) You can access the Project Page by clicking on a “Go to project” button from a Project in the Project List at Consultant Homepage (2) Or access a Project Page with this URL format http://scopecliq.com/domain/<project_id>/.

Actions in the Project Blueprint in Project Page

All the discussion on the Project Blueprint finally comes together in the Project Page.

The Project Page has **2 main purposes**:

1. To design a Project Blueprint that will guide your project to completion
2. To update the Project Blueprint as you progress through your project

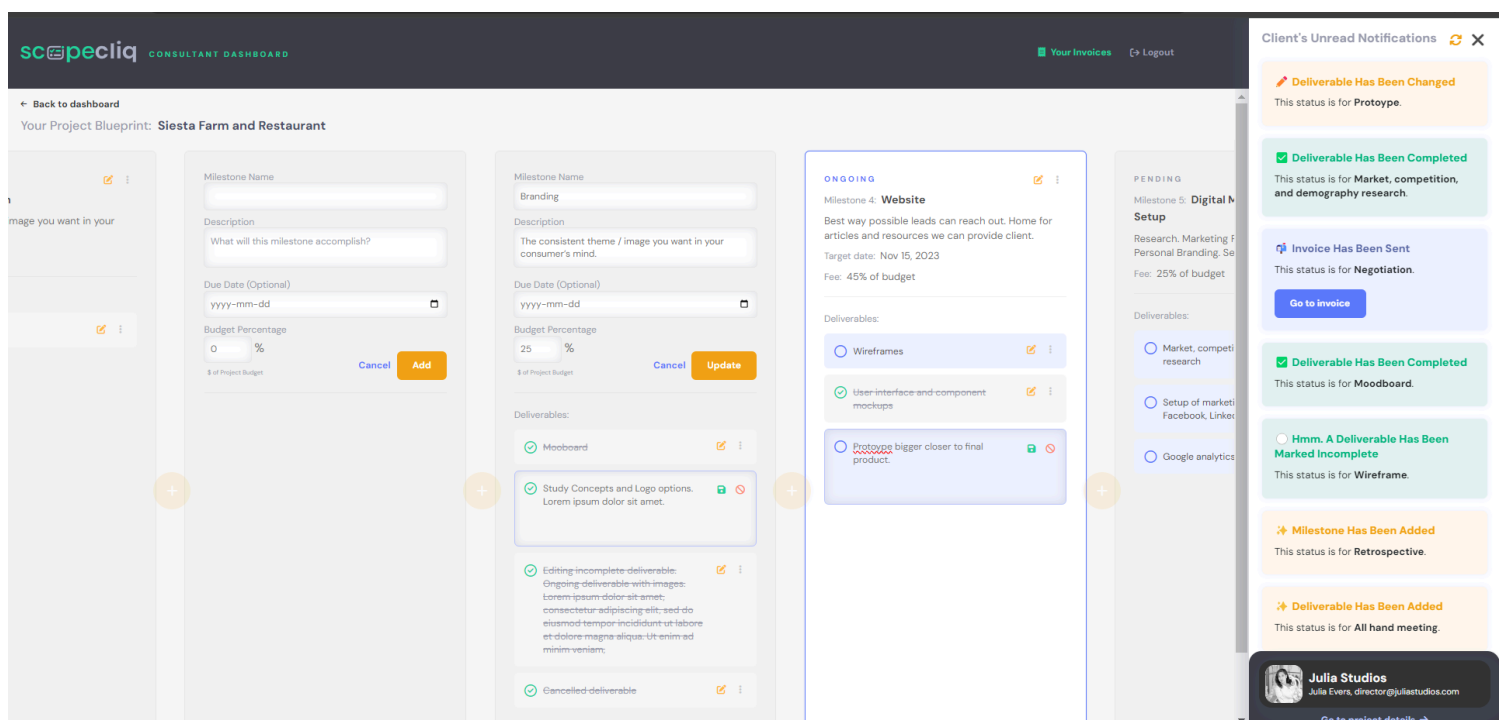


Figure 5.2.1.2. Project Page in action. Editing milestones and deliverables

The quickest thing you can do in the Project Page is to toggle deliverables as Complete or Incomplete. The ease of toggling a Deliverable is essential in the Project Page because its primary objective purpose is to keep the Client updated as you move forward with the project.

In the Project Page you can add, update, and delete a milestone. You can also edit the deliverable description or even delete the deliverable item.

[Creating, Updating, or Deleting a Milestone](#)

[Creating and Updating a Deliverable and Status](#)

Notification Sidebar

Notifications are created when user (1) toggles a Deliverable as Complete or Incomplete, (2) add, update, or delete a Milestone or Deliverable, (3) all Deliverables of a Milestone is Completed marking the whole Milestone as complete.

As a Consultant you will not be able to mark a project's notifications as read. A Notification will only disappear if it marked by your Client as read.

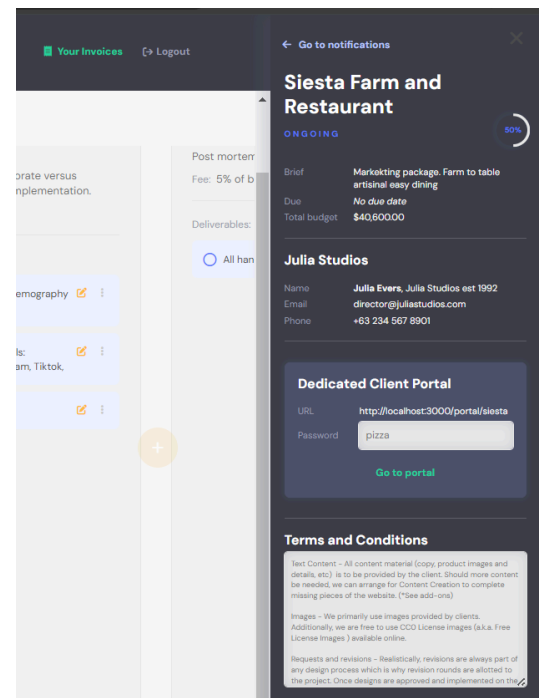
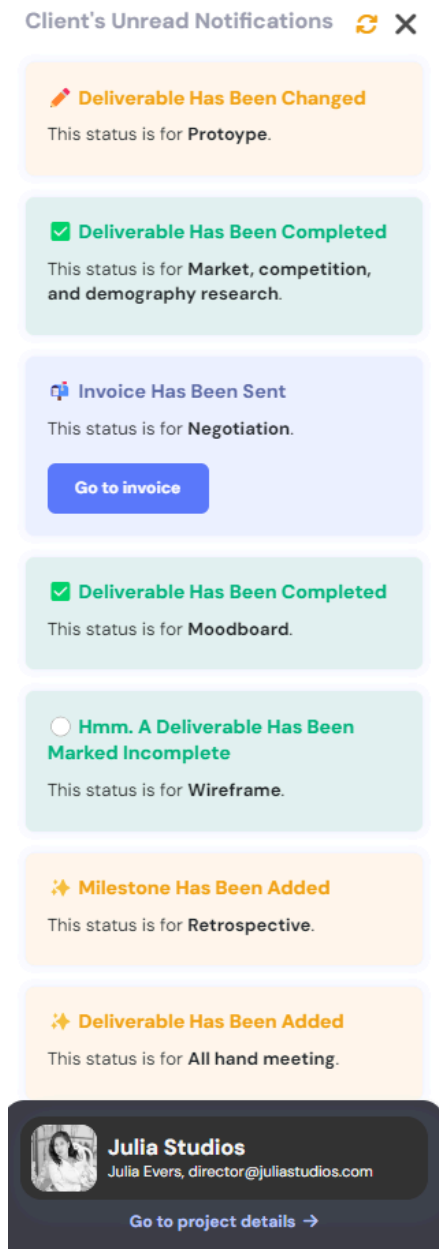
In the Consultant Portal, the sidebar **automatically updates** with new Notifications as you proceed with the actions. This is so you can find a log of which of your work has been acknowledged by your Client.

Figure 5.2.1.3. Consultant Notifications cannot be marked as read

Project Details

Both Client Portal and Consultant Dashboard have the option to view the Project Detail in their sidebars.

Figure 5.2.1.4. Open Project Details (right) by clicking "Go to project details" (left)



2.2 Consultant Dashboard > Consultant Homepage

As a Consultant, you now have a lot of Projects to handle. How do we manage and access all of them?

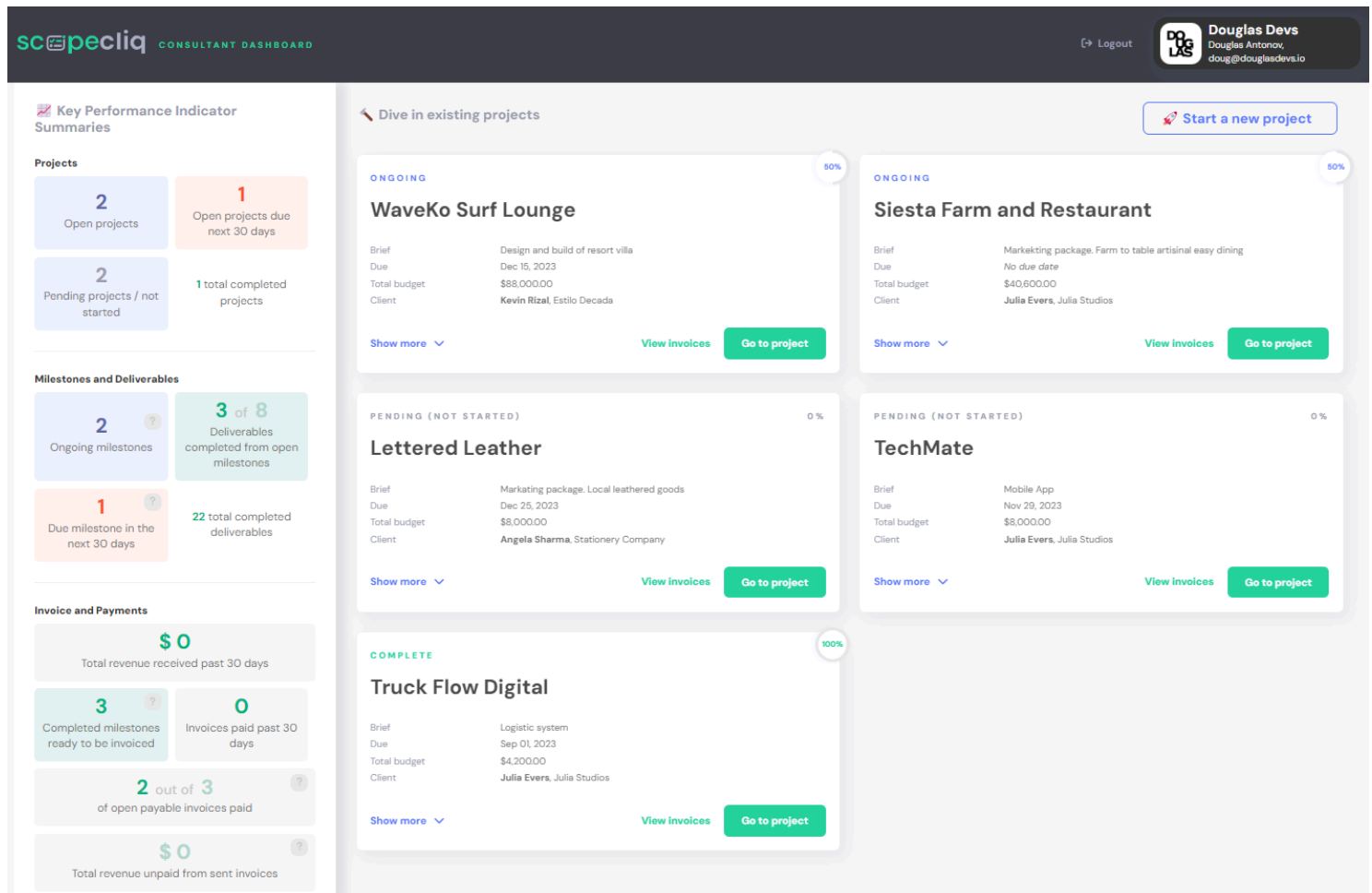


Figure 5.2.1. A closer look at the Consultant Homepage

What is the Consultant Homepage?

The **Consultant Homepage** also holds and organizes all your Projects. From the Consultant Homepage you can check the health of your business. In a glance you can see the progress rate of every project.

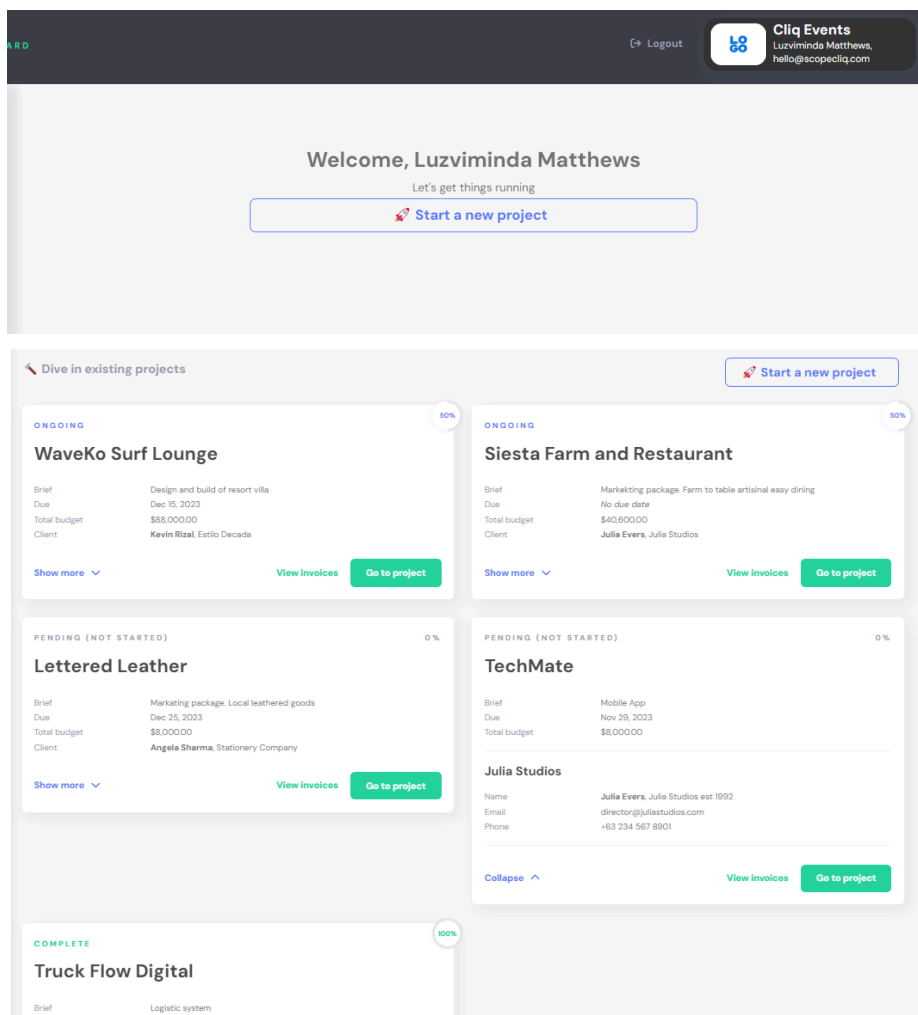
It's two main modules are the **Project List** making the Consultant Homepage your main point to access to all the Projects or create new ones. You also have Key Performance Indicator summaries showing

How to access or navigate to Consultant Homepage?

The Consultant Homepage is called the **Homepage** because when you access our **root url / landing page** – <http://scopecliq.com> – and the browser detects that you are logged in, you will automatically be brought to the Consultant Homepage.

If you are logged in you can click the logo from navigation and it will bring you back to the Consultant Dashboard.

Otherwise, successfully logging in or registering from our **root url / landing page** will also bring you to our Consultant Homepage.



Project List

Figure 5.2.2.2. Empty State of the Project List

You can find all your Projects in the project list with all the essential details such as due date, budget, and client.

Importantly you can find the status and progress rates of the projects to understand if there are any of the projects needing your attention.

You can expand the project cards to show the Client Organization and contact details.

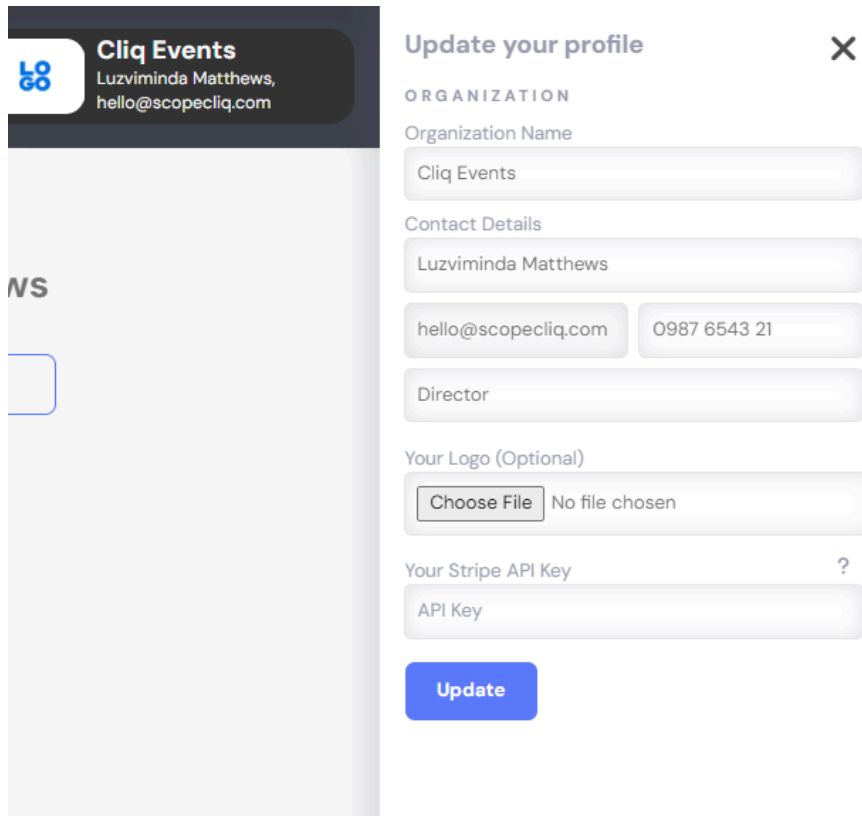
But the full details will be in the sidebar of the Project Page as discussed before.

Figure 5.26 Example of a populated Project List

Adding A New Project

Adding a new project is discussed in the IV.1. Projects section of this report.

[Creating a new Project](#)



The image shows a sidebar titled "Update your profile" with a close button (X) in the top right corner. The sidebar contains the following sections:

- ORGANIZATION**
 - Organization Name: Cliq Events
- Contact Details**
 - Contact Name: Luzviminda Matthews
 - Email: hello@scopecliq.com
 - Phone: 0987 6543 21
 - Job Title: Director
- Your Logo (Optional)**
 - Choose File: No file chosen
- Your Stripe API Key** (with a help icon ?)
 - API Key:
- Update** button

Your Organization Details

You can find your Organization details when you click the **Organization Card** on the upper left corner of the dashboard.

A sidebar for your profile will popup when you can view and edit your information.

Editing your information has **image support** therefore uploading a logo is recommended.

There is also a field for your **Stripe API Key** for invoicing purposes.

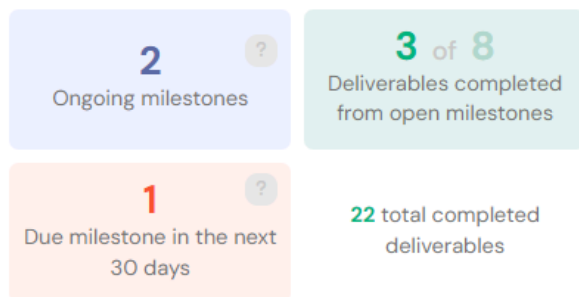
Figure 5.2.2.3. Sidebar for Profile Update

Key Performance Indicator Summaries

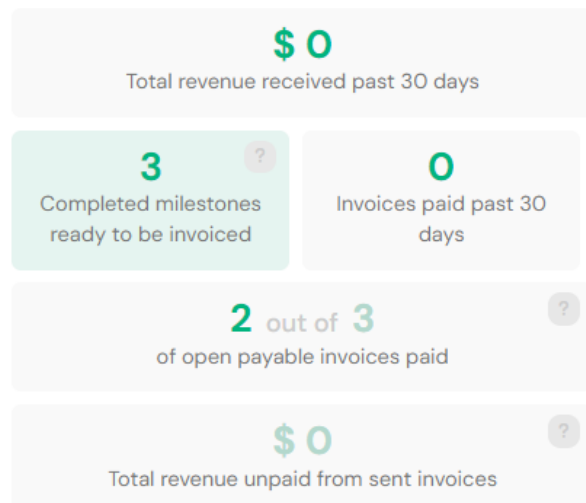
Projects



Milestones and Deliverables



Invoice and Payments



Summaries and Analytics

Since a lot of the Deliverables and Milestones are hidden inside the Project Pages it is essential to have some analytics on the homepage.

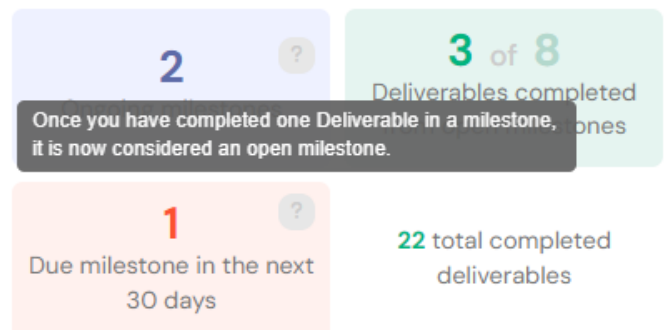
The Analytics are divided by what we consider our KPIs (1) Projects, (2) Milestones and Deliverables, and (3) Payments and Invoices.

Since not everyone is familiar with the terminologies, we employed **tooltips** to offer explanations of the data.

Figure 5.2.2.4. (left) KPIs measured and categorized

Figure 5.2.2.5. (below) Use of tooltip to offer explanations of the data

Milestones and Deliverables



V. Other Features and Functionalities

Notifications

See your project activities and mark read to acknowledge



 **Invoice Has Been Paid** ×

This status is for **Invoice has been paid for invoice #7.**

[Go to invoice](#)

 **Milestone Has Been Completed** ×

This status is for **Prototype.**

 **Deliverable Has Been Completed** ×

This status is for **Wireframes.**

 **Milestone Has Been Changed** ×

This status is for **Digital Marketing Strategy and Setup.**

 **Invoice Has Been Sent** ×

This status is for **Invoice has been created for Retrospective.**

[Go to invoice](#)

 **Deliverable Has Been Completed** ×

This status is for **All hands meeting.**

 **Invoice Has Been Sent** ×

This status is for **Negotiation.**

[Go to invoice](#)

1. Notifications

What is a Notification?

A **Notification** is a log of the activities in the project. The use of Notifications allows Clients to acknowledge the activities completed.

6.1. An array of varying notifications

Types of Notification and Trigger Actions

Status Update Notifications. (1) Whenever a Deliverable changes from Complete and Incomplete; (2) When a Milestone status changes from Pending, Ongoing, or Complete

Change Notifications. When you create, update, or delete a (1) Deliverable or (2) Milestone.

Invoice Notifications. These are triggered when there is any action connected to invoices aka (1) Invoice is generated, (2) Payment is made.

Difference of Notification in Client Portal vs Consultant Dashboard?

For both, we show the similar information in the same area — the Notifications are on the right sidebar.

A **Client** can mark Notifications as read.

As a **Consultant** that option is not available for you and a Notification will only disappear if it is marked by your Client as read. This is so you can find a log of which of your work has been acknowledged by your Client.

In the Consultant Portal, the sidebar is automatically populated with new Notifications as you make changes to the Project Blueprint.

2. Invoices

What is an Invoice?

An Invoice is issued by Consultant to Client once a Milestone is completed. An Invoice item is payable, giving payees access to a payment gateway with correct item information attached. An Invoice has the following properties / information: Invoice ID, total, status, billing date, payment date, **Payment Transaction ID**, payee, receiver, and an **Invoice Link**.

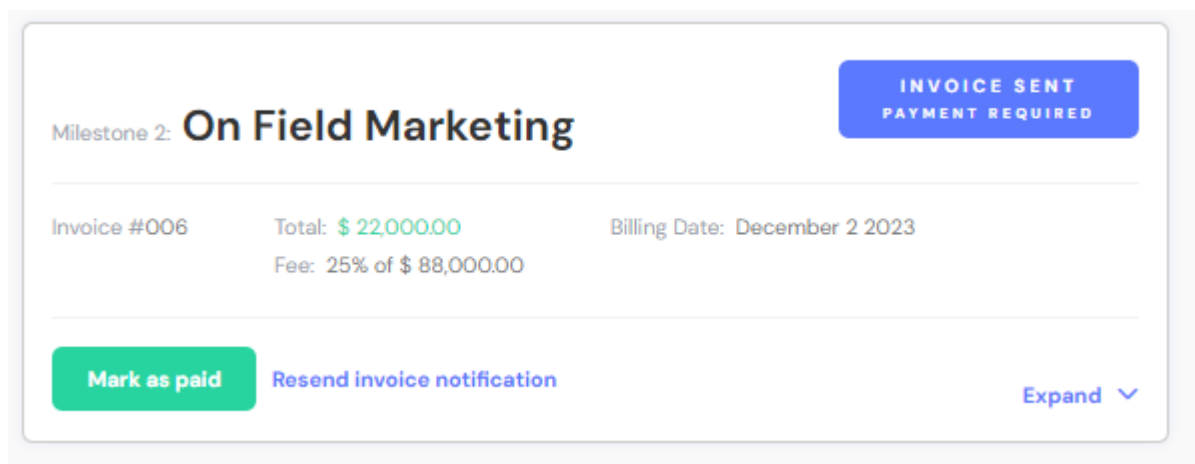


Figure 6.2.1.. Payable Invoice on Consultant Dashboard which show Mark as Paid button

The **possible statuses** for invoice are (1) **Invoice Paid**, (2) **Invoice Sent** / Payment Required, (3) **No Payment Required** (when value of invoice is zero), (4) Unavailable / "There Is No Invoice Yet For This Milestone" (when a Milestone exists but no Invoice has been generated).

An Invoice Card can be Collapsed or Expanded. The Invoice call-to-actions and accessibility are different for a Client versus a Consultant.

A quick **Copy Invoice Link** feature of the invoice list is available when you expand the Invoice Card and click on the Invoice Link input field. This is supported by a Tooltip when you hover.

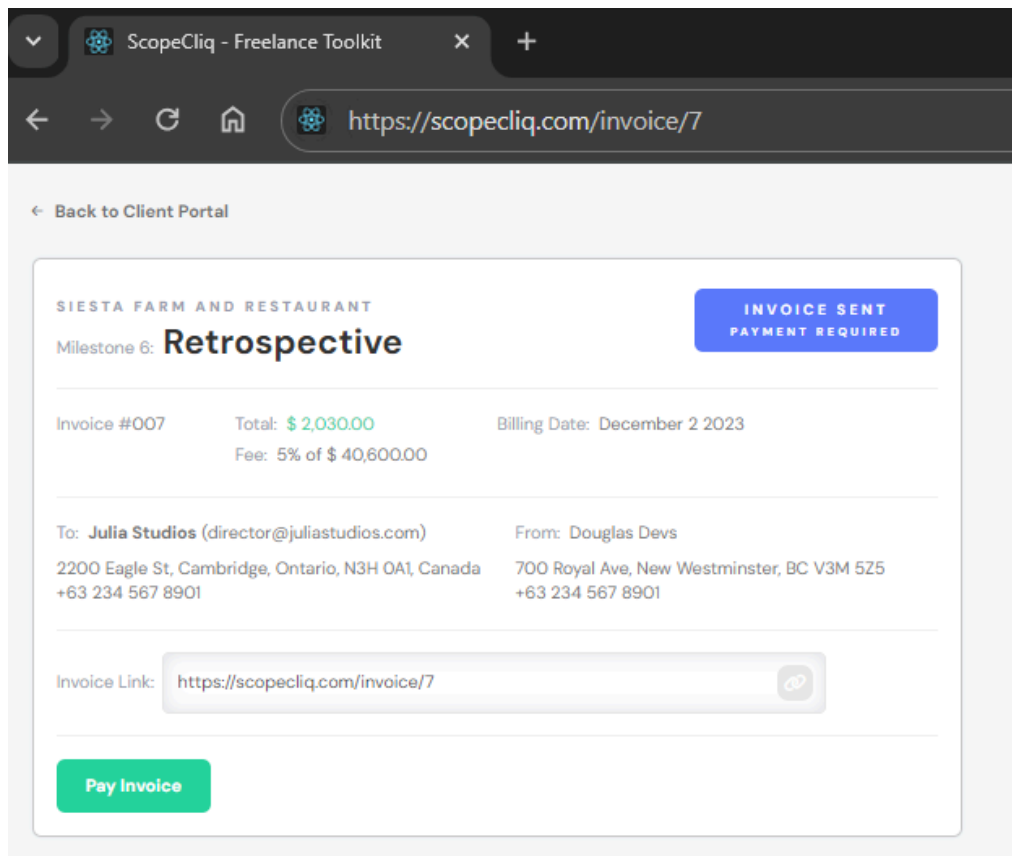


Figure 6.2.2.. Invoice from Client Portal

Global Invoice Link for a Single Invoice Item and Accessibility

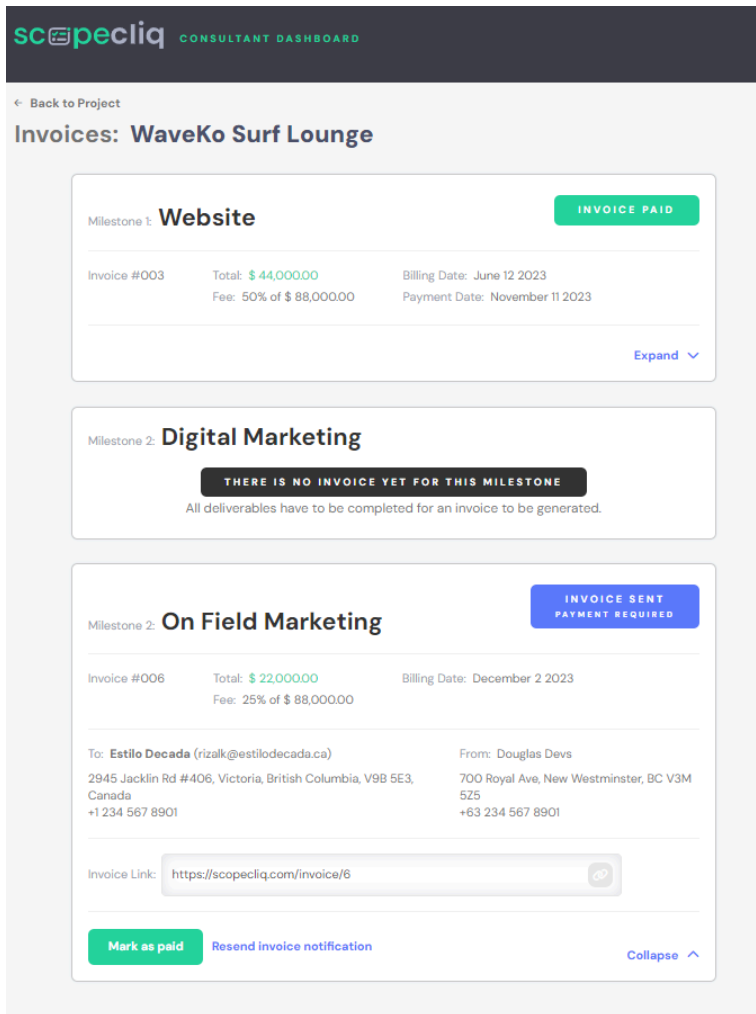
For both Client and Consultant, the route to access an Invoice follows the URL format:
http://scopecliq.com/invoice/<milestone_id>

Anyone can access a single Invoice even if you are not logged in. This design is deliberate as we want anyone to be ABLE to pay an invoice. Such that when you send a link to a higher supervisor they can continue with the payment easily.

You can find that there is no navigation bar at this page as we don't need them navigating anywhere from the page aside from the intent to pay and possibly to go back to the password-protected Client Portal.

Invoice Lists Entry Points and Accessibility

Invoices and Invoice List are available on both Client Portal and Consultant Dashboard. You can find the link to **"View Invoices"** buttons on (1) Client Portal and Consultant Project Page navigation upper right or (2) at the Project's respective Project Card at the Consultant Homepage .



Invoices and Invoice Lists on Client Portal versus Consultant Dashboard

The Invoice looks different for a Client versus a Consultant.

Client Portal Invoices

Call to Actions. You The Invoice in the Client Portal is payable thru our **payment gateway, Stripe.**

Entry Points and Routes. The URL format for Client Portal's Invoice List is

<http://scopecliq.com/portal/>

`<project-domain>/invoices` .

Accessibility. Like the Client Portal, the Invoice List is password-protected for Clients

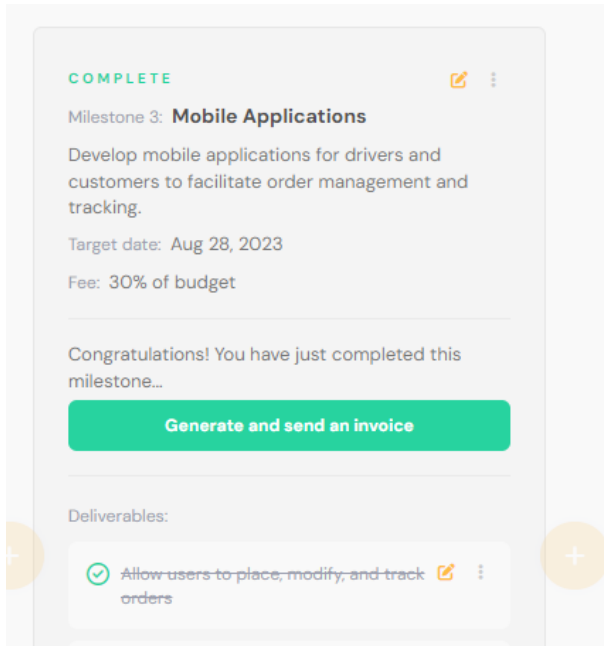
Figure 6.2.3. Invoice List

Consultant Dashboard Invoices

Call to Actions. In the Consultant Dashboard, Consultant can **mark the invoice as paid.** The payment gateway is not necessary in this context. The Consultant can also **resend a notification** of the invoice as a follow up.

Entry Points and Routes. The URL format for Client Portal's Invoice List is http://scopecliq.com/dashboard/<project_id>/invoices .

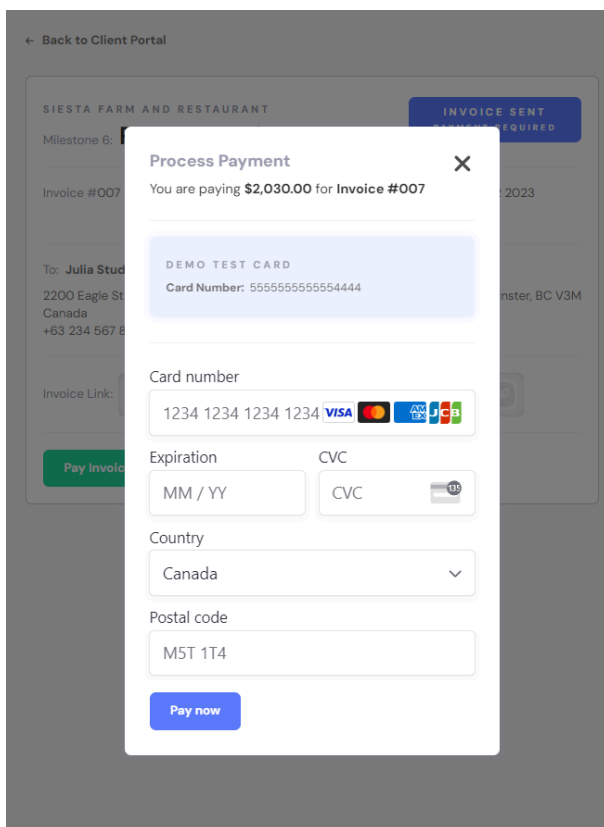
Accessibility. If you are logged in as a Consultant you can view all Invoice Lists without the need to enter the password.



Generating an Invoice

When creating your Milestone, **you assign a percentage of the project budget to the Milestone** in respect to the effort it requires and the value it provides. Once a milestone is completed (all Deliverables marked Complete), the Consultant will be able to generate an Invoice for the client. The **“Generate an Invoice”** button will be shown on the respective milestone at the Consultant Dashboard, which will lead you to a freshly generated Invoice Page in Consultant View.

Figure 6.2.4. “Generate an Invoice” button once the Milestone is completed.



Payments and Stripe Integration

We have integrated Stripe as our platform to process our payment. Stripe was selected because of its seamless application, its security and compliance, and its developer support. Consultant can input their Stripe API Key on the Consultant Homepage Profile Editor.

Upon successful payment, we save data from Stripe’s API successful response to the Invoice item which will mark it as paid with an available Transaction Id.

Figure 6.2.5.. Checkout Modal with Stripe Integration

3. User Management

Consultant versus Client

Only a Consultant has to register to be a User of the website. Clients are not required to sign up to access a project's Client Portal, but it is protected by a password that the Consultant will provide.

As mentioned repeatedly, the Consultant Dashboard and its subpages are only accessible if you are logged in. Client Portal is accessible but blocked by a password.

The Landing Page

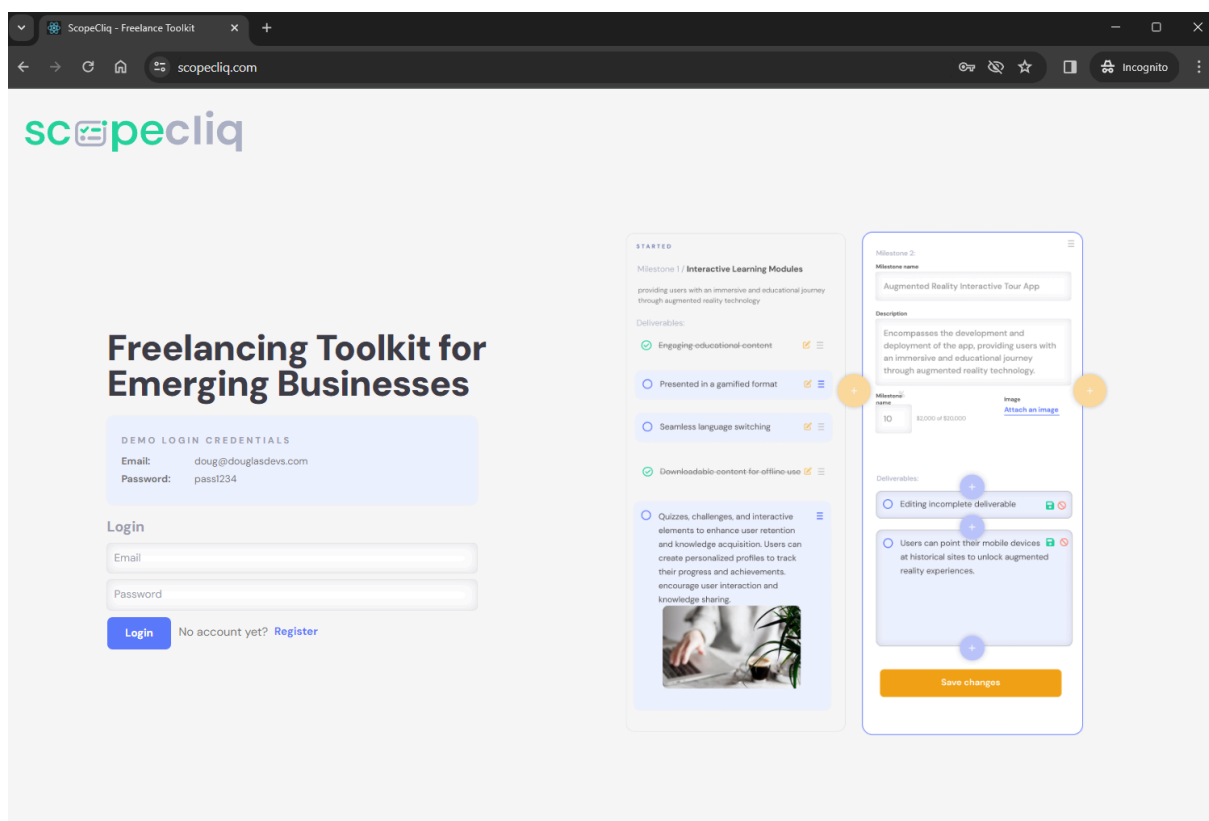


Figure 6.3.1. The Landing Page: <http://scopecliq.com>

When a User goes to <http://scopecliq.com>, if not logged in, the User will be routed to our **Landing Page**. It features a quick value proposition of ScopeCliq as a service and the option to Log in or Register.

This may also be referred to as Homepage if not logged in.

Consultant / The User Object

The Consultant is referred to as a User in our backend. A user has a *user id*, *email*, and *password*.

Login

To login enter your username and password. Demo data is available for the time being as seen on the homepage. For this demo you can use:

Username: doug@douglassdevs.com

Password: pass1234

A snackbar will display if you are unsuccessful in logging in.

Organizations

Organizations are also an object. **Both Clients and Consultant are represented by an Organization object.** An organization has an organization name, contact name, contact, number, contact email, contact about, logo and address.

What differentiates a Client and Consultant in our system is that Consultants Organizations are assigned consultant_user_id which as implied is the id as a user provided once they register.

Freelancing Toolkit for Emergency Business

Tell us more about your business to continue X

YOUR LOGIN DETAILS

Your Email
johnsharma@gmail.com

Your Password

ORGANIZATION

Organization Name
Organization Name

Contact Details
Contact Name
johnsharma@gmail.com Contact Number

Contact About or Role

Register as Consultant

Registration Modal

To register a Consultant inputs their email, password, and password verification. Once the “Register” button is clicked, details are validated.

If valid the registration modal is displayed requiring them to show their password as well. This will prompt User to input the details of their Organization.

From here you can click “Register as Consultant” button to proceed

Figure 6.3.2. Registration

Updating your Account

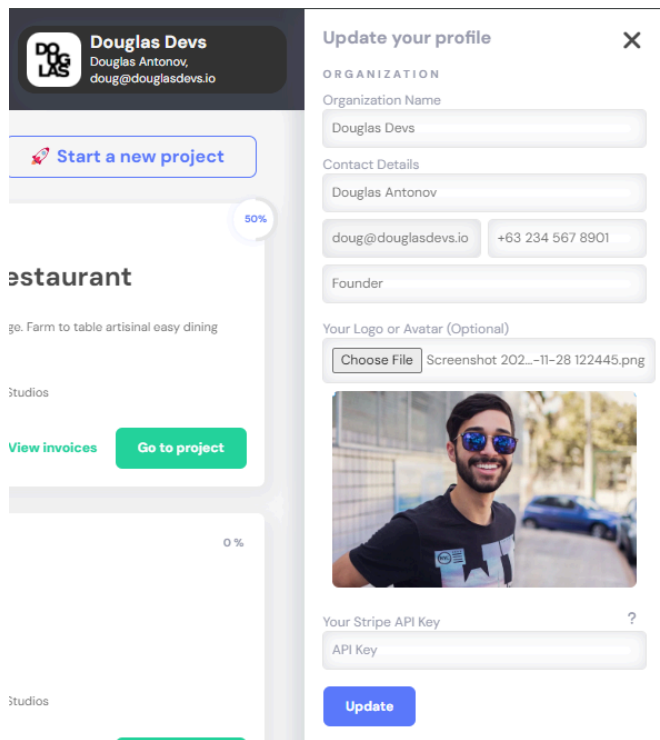
See [Your Organization Details](#)

Uploading a Logo or Avatar

Upload an image to the file input field labelled “Your Logo or Avatar”.

The file will be saved in ScopeCliq’s Cloudinary account then saved to the database once “Update” button is saved.

Figure 6..3.3. Updating your profile on the Consultant Homepage



The screenshot shows a user interface for updating a profile. On the left, a sidebar displays the user's profile information: 'Douglas Devs' with a logo, 'Douglas Antonov', and 'doug@douglasdevs.io'. Below this is a 'Start a new project' button. The main content area shows a project titled 'restaurant' with a description 'ge. Farm to table artisanal easy dining', a 'Studios' section, and buttons for 'View invoices' and 'Go to project'. A progress indicator shows '50%'. On the right, a modal titled 'Update your profile' is open. It contains fields for 'Organization Name' (Douglas Devs), 'Contact Details' (Douglas Antonov, doug@douglasdevs.io, +63 234 567 8901), 'Founder', 'Your Logo or Avatar (Optional)' (with a 'Choose File' button and a file name 'Screenshot 202...-11-28 122445.png'), and 'Your Stripe API Key' (with a question mark icon and a text input field). An 'Update' button is at the bottom of the modal.

VI. Learnings

1. Learnings

The goal of this app is to build a **complete** functional system that blends backend, frontend, operation, and deployment easily.

With all the learning and tools from this CSIS program we were able to blend Laravel and React to build an app locally.

Using whatever SQL practice and theory learned from this year, I was able to translate that in Laravel controllers to fetch tricky data from different tables.

I then was able to use Github for client control and troubleshooting. Github also allowed me to deploy new changes online by pushing my code locally and pulling in production at Siteground.cpm

I was also able to use Figma for good visual architecture by using components to plan and design the states of my objects.

2. Next Phases

It is a goal to scale <http://scopecliq.com> to a professional level. Here are the features that would elevate it to the next level as from our research, surveys, and interview.

White labelling. The Client Portal would be highly elevated if the option to brand it is available.

Chat feature. For real-time Client-Consultant discussions.

Commenting feature. For textual contexts related to an object be it a Project, Milestone, or Deliverable.

Image support. For visual context and interest.

More notification features. Attachment of the object to the Notification. Ex: you will be able to navigate to the actual Deliverable when it is completed.

Activity and logs. A permanent log of activities for accountability.

Project blueprints. The ability to replicate existing successful projects.

VII. Appendix

1. Installation Guide

System Requirements and Technology Stack

These programs are required on your local machine to run the project:

- XAMPP, PHP, MySQL
- Artisan, Composer
- Node.js, React

How to Run this Project

You can download the codebase from: <https://github.com/maccimac/scopecliq>.

Alternatively, the complete codebase is attached in this report:

Project_JMa52913/Implementation/[scopecliq_v1/](#)

```
## Run backend
1. Ensure that XAMPP Apache and PHPMyAdmin are open
2. Ensure your `.env` file is up to date with the correct database username and password.
   ex:
       DB_CONNECTION=mysql
       DB_HOST=127.0.0.1
       DB_PORT=3306
       DB_DATABASE=scopecliq_v1
       DB_USERNAME=root
       DB_PASSWORD=
3. `cd scopecliq-app`
4. `composer install`
5. Run `php artisan migrate --seed` for first time installers, type `y` for yes when prompted to create new db
   or run `php artisan migrate:fresh --seed`
   or import `/scopecliq-app/scopecliq-master-demo.sql`
6. `php artisan serve`
7. Open browser at localhost:8000

## Run frontend
1. `cd scopecliq-ui`
2. `npm run start`
6. Open browser at localhost:3000
```

Also included are directions on how to run in the [README.MD](#)

Troubleshooting

Composer. Commands than you can try for composer issues are:

```
composer dump-autoload  
composer update
```

PHP. Make sure you are running on PHP 8.1 or higher

MySQL and Apache. Make sure your XAMPP is open

Demo data. You can try the following migrate commands:

```
php artisan migrate  
Php artisan migrate --seed  
php artisan migrate:fresh  
php artisan migrate:fresh --seed
```

Ensure your **.env** file has to correct accessible username and password depending on your OS. Try with or without a password. There is an **.env.example**, **.env.local**, and **.env.production** files also for an extra **.env** templates

Demo sql file is also uploaded on `./scopecliq-app/scopecliq-master-demo.sql`

For other issues feel free to email support@scopecliq.com and I will help personally.

2. User Guide

Login and registration

See [Login and Registration](#)

Adding your first project

See [Creating a new Project](#)

Adding your first milestone and deliverables

See [Creating, Updating, or Deleting a Milestone](#)

See [Creating and Updating a Deliverable and Status](#)

Adding your first invoice

See [Adding A New Project](#)

Clients to pay your invoice

See [Payments and Stripe Integration](#)

Updating your profile

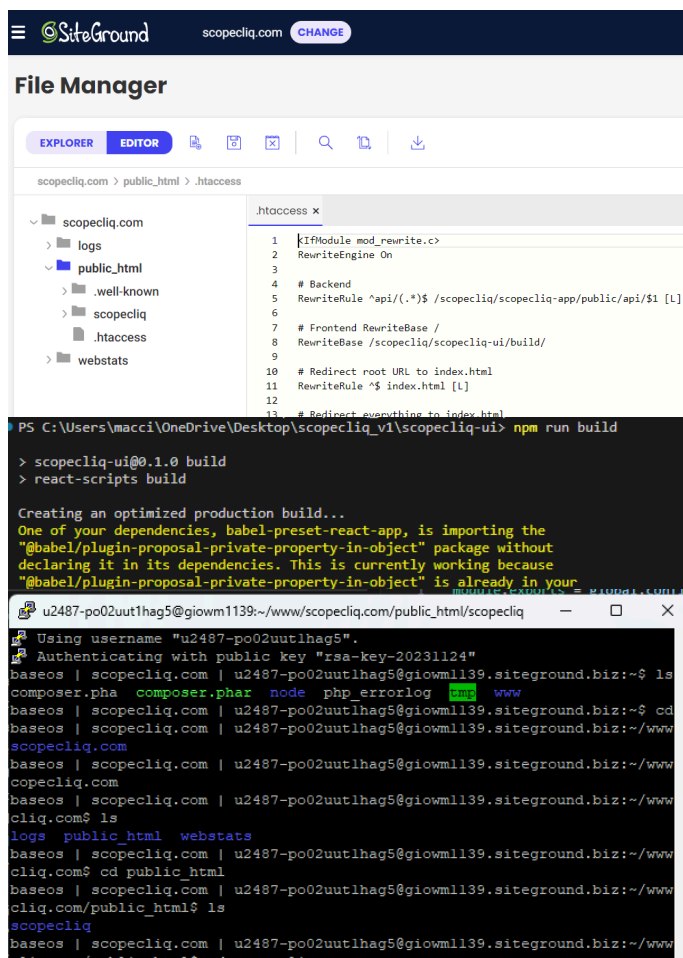
See [Your Organization Details](#)

See [Uploading a Logo or Avatar](#)

3. Architecture

Technology Stack

- **Host:** ScopeCliq is a Web Application
- **Backend Logic and APIs:** Laravel PHP + MySQL, Postman
- **Frontend Interface:** React JS + Node JS + SASS + Bootstrap
- **Deployment and Cloud Services:** Siteground.com, SSH + PuTTY, Cyberduck, Cloudinary
- **IDE:** Visual Studio Code
- **Design:** Figma, Adobe Illustrator



Deployment

Siteground

We invested in a Siteground Hosting to deploy the website online.

Using *npm run build*

For the frontend we used React's *npm run build* command to create optimized build

Using PuTTY and SSH

To access run a Laravel functioning backend in Siteground we had to access the server terminal using SSH and PuTTY (a telnet service)

Figure 9.3.1. Siteground

Figure 9.3.2. *npm run build*

Figure 9.3.3. PuTTY terminal

4. Backend Implementation Codebase

Our backend directory is found in `./scopecliq-app`

4.1. Database Design and Relationships

Design

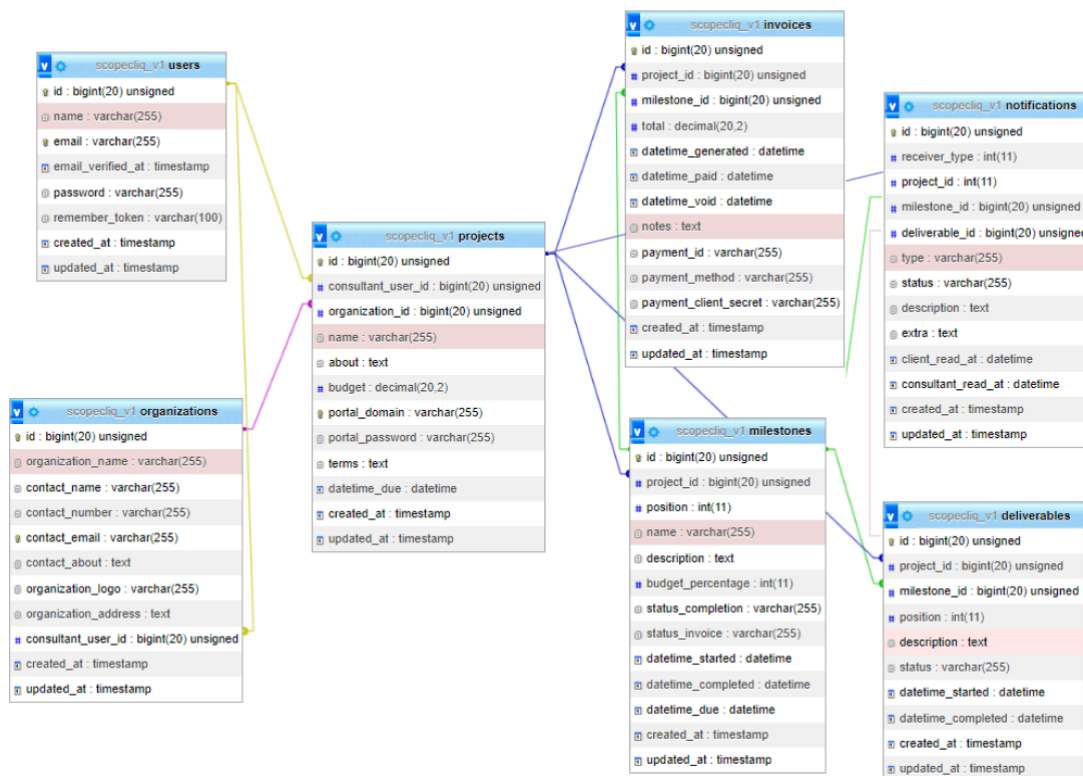


Figure 9.4.0. This is Database Design

Relationships

User – Organization – Project:

A **project** has an **organization_id** (the Client) and a **consultant_user_id** which matches the to a **user** which also has his own **organization**

Project – Milestone – Deliverables:

A **project** has multiple **milestones**. The **milestone** also has multiple **deliverables**. Both **milestone** and **project** can be tracked by their **project_id**.

Project – Milestone – Invoice:

A **milestone** may or may not have an **invoice**. An **invoice** is always attached to a **milestone**. Both are tracked by a **project_id**.

Notifications – Deliverables – Milestones – Invoices – Projects:

A **notification** may or may not be attached to a **deliverable**

A **notification** is attached to a **milestone**. If a **notification_type** is "INVOICE", it will be attached to an **invoice**.

All **notifications** can be tracked by a **project_id**.

4.2. Tables and Migrations

Laravel builds the tables using **migrations**. Below are quick some of the migration codes and

```
C:\Users\macchi\OneDrive\Desktop\scopecliq_v1\scopecliq-app>php artisan migrate:fresh --seed

Dropping all tables ..... 48ms DONE

INFO Preparing database.

Creating migration table ..... 7ms DONE

INFO Running migrations.

2014_10_12_000000_create_users_table ..... 17ms DONE
2014_10_12_100000_create_password_reset_tokens_table ..... 20ms DONE
2019_12_14_000001_create_personal_access_tokens_table ..... 25ms DONE
2023_10_02_044611_create_organizations_table ..... 44ms DONE
2023_10_02_044618_create_projects_table ..... 76ms DONE
2023_10_02_044701_create_milestones_table ..... 35ms DONE
2023_10_15_034015_create_notifications_table ..... 63ms DONE
2023_10_07_012207_create_invoices_table ..... 67ms DONE

INFO Seeding database.

Database\Seeders\UsersTableSeeder ..... RUNNING
Database\Seeders\UsersTableSeeder ..... 1.14 ms DONE

Database\Seeders\OrganizationsTableSeeder ..... RUNNING
Database\Seeders\OrganizationsTableSeeder ..... 1.00 ms DONE

Database\Seeders\ProjectsTableSeeder ..... RUNNING
Database\Seeders\ProjectsTableSeeder ..... 1.24 ms DONE

Database\Seeders\MilestonesTableSeeder ..... RUNNING
Database\Seeders\MilestonesTableSeeder ..... 2.91 ms DONE

Database\Seeders\DeliverablesTableSeeder ..... RUNNING
Database\Seeders\DeliverablesTableSeeder ..... 3.70 ms DONE

Database\Seeders\InvoicesTableSeeder ..... RUNNING
Database\Seeders\InvoicesTableSeeder ..... 1.33 ms DONE

Database\Seeders\NotificationsTableSeeder ..... RUNNING
Database\Seeders\NotificationsTableSeeder ..... 1.45 ms DONE
```

Figure 9.4.2.0.. Migration and Seeding

Migrations

The migration files are attached to the implementation in the following sections.

Figure 9.4.2.1. (left) create_users_table.php; (right) create_organization_table.php

```
public function up(): void
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name')->nullable();
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });
}
```

```
public function up(): void
{
    Schema::create('organizations', function (Blueprint $table) {
        $table->id();

        $table->string('organization_name');
        $table->string('contact_name');
        $table->string('contact_number');
        $table->string('contact_email')->unique();
        $table->text('contact_about');
        $table->string('organization_logo')->nullable();
        $table->text('organization_address')->nullable();

        $table->bigInteger('consultant_user_id')->unsigned()->nullable();
        $table->foreign('consultant_user_id')->references('id')->on('users')->onDelete('');
        $table->timestamps();
    });
}
```

```

public function up(): void
{
    Schema::create('projects', function (Blueprint $table) {
        $table->id();
        $table->bigInteger('consultant_user_id')->unsigned()->nullable();
        $table->foreign('consultant_user_id')->references('id')->on('users');
        $table->foreignId('organization_id')->constrained('organizations');
        $table->string('name');
        $table->text('about')->nullable();
        $table->decimal('budget', 20, 2);
        $table->string('portal_domain')->unique();
        $table->string('portal_password');
        $table->text('terms')->nullable();
        $table->dateTime('datetime_due')->nullable();
        $table->timestamps();
    });
}

```

Figure 9.4.2.3. create_projects_table.php

```

public function up(): void
{
    Schema::create('milestones', function (Blueprint $table) {
        $table->id();
        $table->foreignId('project_id')->constrained('projects');
        $table->integer('position');
        $table->string('name');
        $table->text('description')->nullable();
        $table->integer('budget_percentage');
        $table->string('status_completion')->nullable();
        $table->string('status_invoice')->nullable();
        $table->dateTime('datetime_started')->nullable();
        $table->dateTime('datetime_completed')->nullable();
        $table->dateTime('datetime_due')->nullable();
        $table->timestamps();
    });
}

```

Figure 9.4.2.4. create_milestones_table.php

```

public function up(): void
{
    Schema::create('invoices', function (Blueprint $table) {
        $table->id();
        $table->foreignId('project_id')->constrained('projects');
        $table->foreignId('milestone_id')->constrained('milestones');
        $table->decimal('total', 20, 2);
        $table->dateTime('datetime_generated');
        $table->dateTime('datetime_paid')->nullable();
        $table->dateTime('datetime_void')->nullable();
        $table->text('notes')->nullable();
        $table->string('payment_id')->nullable();
        $table->string('payment_method')->nullable();
        $table->string('payment_client_secret')->nullable();
        $table->timestamps();
    });
}

```

Figure 9.4.2.5. create_invoices_table.php

```

public function up(): void
{
    Schema::create('deliverables', function (Blueprint $table) {
        $table->id();
        $table->foreignId('project_id')->constrained('projects');
        $table->foreignId('milestone_id')->constrained('milestones');
        $table->integer('position');
        $table->text('description');
        $table->string('status')->default('not_started');
        $table->dateTime('datetime_started')->nullable();
        $table->dateTime('datetime_completed')->nullable();
        $table->timestamps();
    });
}

```

Figure 9.4.2.7. create_deliverables_table.php

```

public function up(): void
{
    Schema::create('notifications', function (Blueprint $table) {
        $table->id();
        $table->integer('receiver_type')->default(0); // 0 is consultant,
        $table->integer('project_id');
        $table->bigInteger('milestone_id')->unsigned()->nullable();
        $table->bigInteger('deliverable_id')->unsigned()->nullable();
        $table->string('type');
        // STATUS_UPDATE : {
        //   COMPLETE: `✅` ${attachmentType} has been completed`,
        //   INCOMPLETE: `🟡` Hmm. A ${attachmentType} has been mark
        // },
        // 'INVOICE':{
        //   SENT: `📄 Invoice has been sent`,
        //   PAID: `💰 Invoice has been paid`,
        // },
        // 'CHANGE':{
        //   'MADE': `✏️` ${attachmentType} has been changed`,
        //   'CREATED': `🌟` ${attachmentType} has been added`,
        //   'DELETED': `🗑️` ${attachmentType} deleted`,
        $table->string('status');
        $table->text('description')->nullable();
        $table->text('extra')->nullable();
        $table->dateTime('client_read_at')->nullable();
        $table->dateTime('consultant_read_at')->nullable();
        $table->timestamps();
    });
}

```

Figure 9.4.2.6. Create_notifications_table.php
(with comments of which values to use)

```
scopecliq-app > database > seeders > DatabaseSeeder.php
1  <?php
2
3  namespace Database\Seeders;
4  use Illuminate\Database\Seeder;
5  use Illuminate\Support\Facades\DB;
6
7  class DatabaseSeeder extends Seeder
8  {
9      /**
10       * Seed the application's database.
11       */
12       public function run(): void
13       {
14           $this->call(UsersTableSeeder::class);
15           $this->call(OrganizationsTableSeeder::class);
16           $this->call(ProjectsTableSeeder::class);
17           $this->call(MilestonesTableSeeder::class);
18           $this->call(DeliverablesTableSeeder::class);
19           $this->call(InvoicesTableSeeder::class);
20           $this->call(NotificationsTableSeeder::class);
21       }
22 }
```

4.3. Seeders

Laravel allowed the use of Seeders to populate the database with data of correct type and relationship.

Figure 9.4.3.1. DatabaseSeeder.php did all the magic of running our seeder data

Below are some of our seeder files. At this point these names and descriptions might be familiar.

Figure 9.4.3.2. ProjectsTableSeeder.php, UsersTableSeeder.php, OrganizationsTableSeeder.php

```
DB::table('projects')->insert([
    [
        'organization_id' => 1,
        'name' => 'WaveKo Surf Lounge',
        'about' => 'Design and build of resort villa',
        'budget' => 88000,
        'portal_domain' => 'waveko',
        'portal_password' => 'beach',
        'datetime_due' => Carbon::create(2023, 12, 15, 0, 0, 0),
        'consultant_user_id' => 1,
        'terms' => $termsPlaceholder,
    ],
    [
        'organization_id' => 2,
        'name' => 'Siesta Farm and Restaurant',
        'about' => 'Marketing package. Farm to table artisanal e',
        'budget' => 40600,
        'portal_domain' => 'siesta',
        'portal_password' => 'pizza',
        'datetime_due' => null,
        'consultant_user_id' => 1,
        'terms' => $termsPlaceholder,
    ],
]);
```

```
public function run(): void
{
    DB::table('users')->insert([
        [
            'name' => 'Douglas Dev',
            'email' => 'doug@douglasdevs.com',
            'password' =>
                '$2y$10$ePeR9rh9pEIqLHL7LM0BnuESGMjUIMiExP37SqN9bwe2jcMWF6BK', // pass1234
        ],
    ]);
}
```

```
DB::table('organizations')->insert([
    [
        'organization_name' => 'Estilo Decada',
        'contact_name' => 'Kevin Rizal',
        'contact_email' => 'rizalk@estilodecada.ca',
        'contact_about' => 'Head Contractor at Estilo Decada',
        'contact_number' => '+1 234 567 8901',
        'organization_address' => '2945 Jacklin Rd #406, Victoria, British Columbia, V9B 5E3, Canada',
        'organization_logo' => null,
        'consultant_user_id' => null
    ],
    [
        'organization_name' => 'Julia Studios',
        'contact_name' => 'Julia Evers',
        'contact_email' => 'director@juliastudios.com',
        'contact_about' => 'Julia Studios est 1992',
        'contact_number' => '+63 234 567 8901',
        'organization_address' => '2200 Eagle St, Cambridge, Ontario, N3H 0A1, Canada',
        'organization_logo' => 'https://res.cloudinary.com/dtvs2pru/image/upload/v1701369706/kxvvnsey2gfucxdyo6pq.jpg',
        'consultant_user_id' => null
    ],
    [
        'organization_name' => 'Stationery Company',
        'contact_name' => 'Angela Sharma',
        'contact_email' => 'angela@office.com',
        'contact_about' => 'Accounts Manager',
        'contact_number' => '+1 234 567 8901',
        'organization_address' => '2945 Jacklin Rd #406, Victoria, British Columbia, V9B 5E3, Canada',
        'organization_logo' => null,
        'consultant_user_id' => null
    ],
    [
        'organization_name' => 'Douglas Devs',
        'contact_name' => 'Douglas Antonov',
        'contact_email' => 'doug@douglasdevs.io',
        'contact_about' => 'Founder',
        'contact_number' => '+63 234 567 8901',
        'organization_address' => '700 Royal Ave, New Westminster, BC V3M 5Z5',
        'organization_logo' => 'https://pbs.twimg.com/profile_images/1717259548569075712/X0Up3325_400x400.jpg',
        'consultant_user_id' => 1
    ],
]);
```

4.4. Routes and APIs

api.php

Our **api.php** controlled all routes we required to fetch organized data from our database. The sections are labelled properly and the names are relevant to the data being fetched or being transformed.

```
// USERS
Route::prefix('user')->group(function () {
    Route::post('/login', [UserController::class, 'login']);
    Route::post('/register', [UserController::class, 'register']);
    Route::post('/validate', [UserController::class, 'validateRegistration']);
});

// ORGANIZATIONS
Route::prefix('organizations')->group(function () {
    Route::get('/', [OrganizationsController::class, 'fetchAllOrganizations']);
    Route::get('/{organization_id}', [OrganizationsController::class, 'fetchOrganizationById']);
    Route::get('/consultant/{user_id}', [OrganizationsController::class, 'fetchOrganizationByConsultantId']);
    Route::post('/add/{user_id}', [OrganizationsController::class, 'addOrganization']);
    Route::post('/update/{organization_id}', [OrganizationsController::class, 'updateOrganizationById']);
    Route::post('/update-logo/{organization_id}', [OrganizationsController::class, 'updateOrganizationLogoById']);
});

// PROJECTS
Route::prefix('projects')->group(function () {
    Route::get('/', [ProjectsController::class, 'fetchAllProjects']);
    Route::get('/consultant-user-id/{user_id}', [ProjectsController::class, 'fetchAllProjectsByConsultantUserId']);
    Route::get('/{project_id}', [ProjectsController::class, 'fetchById']);
    Route::get('/portal/{portal}', [ProjectsController::class, 'fetchByPortal']);
    Route::get('/organization/{organization_id}', [ProjectsController::class, 'fetchProjectsByOrganization']);
    Route::get('/consultant/{consultant_id}', [ProjectsController::class, 'fetchProjectsByConsultant']);
    Route::post('/add/{organization_id}', [ProjectsController::class, 'addProject']);
});

// MILESTONES
Route::prefix('milestones')->group(function () {
    Route::get('/{milestone_id}', [MilestonesController::class, 'fetchMilestoneById']);
    Route::get('/project/{project_id}', [MilestonesController::class, 'fetchMilestonesByProject']);
    Route::get('/last-position/{project_id}', [MilestonesController::class, 'getLastPositionByProject']);
    Route::post('/add/{project_id}', [MilestonesController::class, 'addMilestoneToProject']);
    Route::post('/update/{milestone_id}', [MilestonesController::class, 'updateMilestoneById']);
    Route::post('/update-position/{milestone_id}/{position_id}', [MilestonesController::class, 'updateMilestonePositionById']);
    Route::post('/update-status/{milestone_id}/{status}', [MilestonesController::class, 'updateMilestoneStatusById']);
    Route::post('/delete/{milestone_id}', [MilestonesController::class, 'deleteMilestoneById']);
});
```

Figure 9.4.4.1. api.php 1 of 2: Users, Organizations, Projects, and Milestones

```

// DELIVERABLES
Route::prefix('deliverables')->group(function () {
    Route::get('/{id}', [DeliverablesController::class, 'fetchDeliverableById']);
    Route::get('/project/{project_id}', [DeliverablesController::class, 'fetchDeliverablesByProject']);
    Route::get('/milestone/{milestone_id}', [DeliverablesController::class, 'fetchDeliverablesByMilestone']);
    Route::post('/update/{id}/status/{status}', [DeliverablesController::class, 'updateDeliverableStatus']);
    Route::post('/update/{id}/position/{position}', [DeliverablesController::class, 'updateDeliverablePosition']);
    Route::post('/add/milestone/{milestone_id}', [DeliverablesController::class, 'addDeliverableToMilestone']);
    Route::post('/edit/{id}', [DeliverablesController::class, 'editDeliverableById']);
    Route::post('/delete/{id}', [DeliverablesController::class, 'deleteDeliverableById']);
});

// INVOICES
Route::prefix('invoices')->group(function () {
    Route::get('/{id}', [InvoicesController::class, 'fetchInvoiceById']);
    Route::post('/milestone/{milestone_id}', [InvoicesController::class, 'fetchInvoiceByMilestoneIdFull']);
    Route::get('/project/{project_id}', [InvoicesController::class, 'fetchInvoicesByProject']);
    Route::post('/mark-paid/{id}', [InvoicesController::class, 'markInvoicePaid']);
    Route::post('/pay/{id}', [InvoicesController::class, 'payInvoice']);
    Route::post('/create', [InvoicesController::class, 'createInvoiceOfMilestone']);
});

// NOTIFICATIONS
Route::prefix('notifications')->group(function () {
    Route::get('/{id}', [NotificationsController::class, 'fetchNotificationById']);
    Route::get('/project/{project_id}', [NotificationsController::class, 'fetchNotificationsByProject']);
    Route::post('/project/{project_id}/add', [NotificationsController::class, 'addNotificationToProject']);
    Route::post('/read/{id}', [NotificationsController::class, 'markNotificationAsRead']);
});

// ANALYTICS
Route::prefix('analytics')->group(function () {
    Route::get('/project/{project_id}/progress', [AnalyticsController::class, 'fetchProgressPercentByProject']);
    Route::get('/milestone/{milestone_id}/progress', [AnalyticsController::class, 'fetchProgressPercentByMilestone']);
    Route::get('/{user_id}/projects', [AnalyticsController::class, 'fetchProjectsAnalytics']);
    Route::get('/{user_id}/milestones', [AnalyticsController::class, 'fetchMilestonesAnalytics']);
    Route::get('/{user_id}/invoices', [AnalyticsController::class, 'fetchInvoicesAnalytics']);
    Route::get('/{user_id}/match-milestone-to-invoice', [AnalyticsController::class, 'matchMilestonesToInvoice']);
});

```

Figure 9.4.4.2. api.php 2 of 2: Deliverables, Invoices, Notifications, and Analytics

4.5. Controllers

These controllers handle the requests from above `api.php`. Once called from **api.php**, the respective **<Object>Controller.php** processes the data by interacting with our MySQL database. Basically these are the hardworking middlemen between the api calls and the database.

User Controller

Login and Registration for Users. Special **validateRegistration** was also created for when we want to allidant without automatically registering.

```
class UserController extends Controller
{
    public function register(Request $request)
    {
        // Implement your custom registration logic here.
        // Create a new user, validate input, and save to the database.

        $this->validate($request, [
            'email' => 'required|email|unique:users',
            'password' => 'required|string|min:8',
        ]);

        $user = new User;
        $user->name = $request->input('name');
        $user->email = $request->input('email');
        $user->password = bcrypt($request->input('password'));
        $user->save();

        return response()->json([
            'status' => 'success',
            'message' => 'Registration successful',
            'user_id' => $user->id
        ]);
    }
}
```

```
public function validateRegistration(Request $request){
    $validatedData = $this->validate($request, [
        // 'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|string|min:8',
    ]);

    return response()->json(['message' => 'Validation successful', 'data' =>
        $validatedData, 'status' => 'success']);
}

public function login(Request $request)
{
    // Implement your custom authentication logic here.
    // Check the user's credentials, generate a token, or return a JSON response.

    if (auth()->attempt(['email' => $request->input('email'), 'password' =>
        $request->input('password')])) {
        // Authentication successful
        $user = auth()->user();
        return response()->json([
            'message' => 'Login successful',
            'status' => 'success',
            'user_id' => $user->id
        ]);
    } else {
        // Authentication failed
        return response()->json([
            'message' => 'Login failed',
            'status' => 'error'
        ], 401); // You can adjust the status code as needed.
    }
}
```

Figure 9.4.5.1. UserController.php

Organization Controller

Basic CRUD for Organization table / object. Figure 9.4.5.2. OrganizationsController.php

```
class OrganizationsController extends Controller
{
    //
    public function fetchAllOrganizations(){
        $clients = DB::table('organizations')
            -> select('*')
            -> get();
        return $clients;
    }

    public function fetchOrganizationById($organization_id){
        $clients = DB::table('organizations')
            -> select('*')
            -> where('id', $organization_id)
            -> first();
        return $clients;
    }

    public function fetchOrganizationByConsultantId($user_id){
        $clients = DB::table('organizations')
            -> select('*')
            -> where('consultant_user_id', $user_id)
            -> first();
        return $clients;
    }

    public function addOrganization(Request $req, $user_id) {
        $newOrgId = DB::table('organizations')
            -> insertGetId([
                'organization_name' => $req->organization_name,
                'contact_name' => $req->contact_name,
                'contact_email' => $req->contact_email,
                'contact_about' => $req->contact_about,
                'contact_number' => $req->contact_number,
                'consultant_user_id' => $user_id,
            ]);
        return $newOrgId;
    }
}
```

```
public function addOrganization(Request $req, $user_id) {
    $newOrgId = DB::table('organizations')
        -> insertGetId([
            'organization_name' => $req->organization_name,
            'contact_name' => $req->contact_name,
            'contact_email' => $req->contact_email,
            'contact_about' => $req->contact_about,
            'contact_number' => $req->contact_number,
            'consultant_user_id' => $user_id,
        ]);
    return $newOrgId;
}

public function updateOrganizationById (Request $req, $organization_id) {
    $data = [
        'organization_name' => $req->organization_name,
        'contact_name' => $req->contact_name,
        'contact_email' => $req->contact_email,
        'contact_about' => $req->contact_about,
        'contact_number' => $req->contact_number,
        'organization_logo' => $req->organization_logo
    ];

    $organization = DB::table('organizations')
        -> where('id', $organization_id)
        -> update($data);

    return $organization;
}

public function updateOrganizationLogoById (Request $req, $organization_id) {
    $data = [
        'organization_logo' => $req->organization_logo
    ];

    $organization = DB::table('organizations')
        -> where('id', $organization_id)
        -> update($data);

    return $organization;
}
```


Project Controller

Project Controller has our basic CRUD but in many cases filtered by Consultant requirement versus Client requirements.

```
class ProjectsController extends Controller
{
    //

    public function fetchAllProjects(){
        $projects = DB::table('projects')
            -> select('*')
            -> get();
        return $projects;
    }

    public function fetchAllProjectsByConsultantUserId($user_id){
        $projects = DB::table('projects')
            -> select('*')
            -> where('consultant_user_id', $user_id)
            -> get();
        return $projects;
    }

    public function fetchById($project_id){
        $project = DB::table('projects')
            -> select('*')
            -> where('id', $project_id)
            -> first();
        return $project;
    }

    public function fetchByPortal($portal){
        $projects = DB::table('projects')
            -> select('*')
            -> where('portal_domain', $portal)
            -> first();
        return $projects;
    }

    public function fetchProjectsByOrganization($organization_id){
        $projects = DB::table('projects')
            -> select('*')
            -> where('organization_id', $organization_id)
            -> get();
        return $projects;
    }
}
```

```
public function fetchProjectsByConsultant($consultant_id){
    /* ...
    */

    $projects = DB::table('projects')
        -> select('*')
        -> join('organizations', 'projects.organization_id', '=', 'organizations.id')
        -> where('consultant_id', '=', $consultant_id)
        -> get();

    return $projects;
}

public function addProject(Request $req, $organization_id) {

    $this->validate($req, [

        'name' => 'required|string',
        'portal_domain' => 'required|string|unique:projects',
        'portal_password' => 'required|string',
        'datetime_due' => 'nullable|date|after:now', // Ensures datetime_due
        is in the future
        'consultant_user_id' => 'required|numeric',

    ]);

    $newProjId = DB::table('projects')
        -> insertGetId([

            'organization_id' => $organization_id,
            'name' => $req->name,
            'budget' => $req->budget,
            'about' => $req->about,
            'status' => 'pending',
            'portal_domain' => $req->portal_domain,
            'portal_password' => $req->portal_password,
            'datetime_due' => $req->datetime_due,
            'terms' => $req->terms,
            'consultant_user_id' => $req->consultant_user_id

        ]);

    return $newProjId;
}
```

Figure 9.4.5.3. ProjectsController.php

Milestone Controller

Milestone CRUD. Special mention here would be **updateMilestonePositionById** ID which helped a lot when adding a Milestone between 2 Milestones. We also have **updateMilestoneStatusById** for when the statuses of deliverables match the condition that a Milestone should be Pending, Ongoing, or Complete.

```
class MilestonesController extends Controller
{
    //

    public function fetchMilestonesByProject($project_id){
        $projects = DB::table('milestones')
            -> select('*')
            -> where('project_id', $project_id)
            -> orderBy('position')
            -> get();
        return $projects;
    }

    public function fetchMilestoneById($milestone_id){
        $projects = DB::table('milestones')
            -> select('*')
            -> where('id', $milestone_id)
            -> get();
        return $projects;
    }

    public function deleteMilestoneById($milestone_id){
        $deliverablesCount = DB::table('deliverables')
            -> where('milestone_id', $milestone_id)
            -> count();

        if($deliverablesCount > 0){
            throw new \Exception("Cannot delete milestone with deliverables. Delete deliverables first.");
        }

        $deleted = DB::table('milestones')
            -> where('id', $milestone_id)
            -> delete();

        if ($deleted) {
            // Optionally, you can return a success message or a response here
            return [
                'status' => 'success',
                'message' => "Milestone with ID $milestone_id has been deleted.",
                'milestone_id' => $milestone_id
            ];
        } else {
            return [
                'status' => "fail",
                'message' => "Milestone with ID $milestone_id not found."
            ];
        }
    }

    public function updateMilestonePositionById (Request $req, $milestone_id, $position) {
        $data = [
            'position' => $position
        ];

        $milestone = DB::table('milestones')
            -> where('id', $milestone_id)
            -> update( $data );
        return $milestone;
    }

    public function updateMilestoneStatusById (Request $req, $milestone_id, $status)
    {
        $data = [
            'status_completion' => $status
        ];

        $milestone = DB::table('milestones')
            -> where('id', $milestone_id)
            -> update( $data );

        $milestone = DB::table('milestones')
            -> where('id', $milestone_id)
            -> first();

        return $milestone;
    }

    static public function getLastPositionByProject($project_id){
        /*
        SELECT position
        FROM `milestones`
        where project_id = 2
        order by position desc
        limit 1;
        */

        $result = DB::table('milestones')
            ->select('position')
            ->where('project_id', $project_id)
            ->orderByDesc('position')
            ->limit(1)
            ->value('position');

        return $result;
    }

    public function addMilestoneToProject(Request $req, $project_id) {

        // get last position
        $lastPosition = $this -> getLastPositionByProject($project_id);

        $newId = DB::table('milestones')
            ->insertGetId([
                'project_id' => $project_id,
                'position' => $lastPosition+1,
                'name' => $req->name,
                'description' => $req->description,
                'budget_percentage' => $req->budget_percentage,
                'datetime_due' => $req->datetime_due,
                'status_completion' => null,
                'status_invoice' => null,
                'datetime_started' => null,
            ]);

        return $newId;
    }

    public function updateMilestoneById (Request $req, $milestone_id) {
        $data = [
            'name' => $req->name,
            'description' => $req->description,
            'budget_percentage' => $req->budget_percentage,
            'datetime_due' => $req->datetime_due,
        ];

        $milestone = DB::table('milestones')
            -> where('id', $milestone_id)
            -> update( $data );
        return $milestone;
    }
}
```

Figure 9.4.5.4. MilestonesController.php

Deliverables Controller

Basic CRUD for deliverables. Most often used are (1) **updateDeliverableStatus** for toggling the controllers, (2) **updateDeliverablePosition** for updating all the positions of deliverables in a milestone when adding or deleting between deliverables, (3) **fetchDeliverableByMilestoneId** and **fetchDeliverableByProject**, and (4) **addDeliverableToMilestone**.

```
class DeliverablesController extends Controller
{
    public function fetchDeliverableById($id){
        $deliverable = DB::table('deliverables')
        -> select('*')
        -> where('id', $id)
        -> first();

        return $deliverable;
    }

    public function fetchDeliverablesByProject($project_id){
        $deliverables = DB::table('deliverables')
        -> select('*')
        -> where('project_id', $project_id)
        -> get();
        return $deliverables;
    }

    public function fetchDeliverablesByMilestone($milestone_id){
        $deliverables = DB::table('deliverables')
        -> select('*')
        -> where('milestone_id', $milestone_id)
        -> orderBy('position')
        -> get();
        return $deliverables;
    }

    public function updateDeliverableStatus($id, $status) {

        $deliverable = DB::table('deliverables')
        -> where('id', $id)
        -> update([
            'status'=> $status
        ]);
        return $deliverable;
    }

    public function deleteDeliverableById($id) {

        $deliverable = DB::table('deliverables')
        -> where('id', $id)
        -> delete();
    }
}
```

```
public function addDeliverableToMilestone(Request $req, $milestone_id){
    $lastPosition = $this -> getLastPositionByMilestone($milestone_id);
    $projId = DB::table('milestones')
    ->select('project_id')
    ->where('id', $milestone_id)
    ->limit(1)
    ->value('project_id');

    $newDeliverable = DB::table('deliverables')
    ->insert([
        [
            'project_id'=> $projId,
            'milestone_id' => $milestone_id,
            'position' => $req->position,
            'status'=> 'INCOMPLETE',
            'description' => $req->description,
        ],
    ]);
    $lastInsertId = DB::getPdo()->lastInsertId();
    $deliverable = DB::table('deliverables')
    -> where('id', $lastInsertId)
    -> first();
    return $deliverable;
}
```

```
public function updateDeliverablePosition($id, $position) {

    $deliverable = DB::table('deliverables')
    -> where('id', $id)
    -> update([
        'position'=> $position
    ]);
    // $deliverable;

    $deliverable = DB::table('deliverables')
    -> where('id', $id)
    -> first();
    return $deliverable;
}

public function editDeliverableById(Request $req, $id) {

    $d = [];
    if($req->description){ $d['description'] = $req->description;}
    if($req->status){ $d['status'] = $req->status; }

    $deliverable = DB::table('deliverables')
    -> where('id', $id)
    -> update( $d );
    return $deliverable;
}
```

Figure 9.4.5.5. DeliverablesController.php

Invoices Controller

These are the controllers for:

- (1) **createInvoiceOfMilestones** generating invoice in respect to the milestone it is attached to,
- (2) **fetchInvoiceById** versus **fetchInvoiceByIdFull** (The latter having more exhaustive information),
- (3) **markInvoiceAsPaid** (where we just add payment date) versus **payInvoice** (where information from Stripe payment is also recorded as **payment_client_secret**).

```
class InvoicesController extends Controller
{
    public function fetchInvoicesByProject($project_id){
        $invoices = DB::table('invoices as i')
            ->select(
                'i.id',
                'i.total',
                'i.project_id',
                'i.milestone_id',
                'i.datetime_generated',
                'i.datetime_paid',
                'i.datetime_void',
                'i.notes',
                'i.payment_id',
                'i.payment_id',
                'm.position',
            )
            ->join('milestones as m', 'i.milestone_id', '=', 'm.id')
            ->where('i.project_id', $project_id)
            ->get();

        return $invoices;

        // select i.id, i.total, i.project_id, i.milestone_id, i.datetime_generated,
        // i.datetime_paid, i.datetime_void, i.notes, m.position from invoices as i
        // inner join milestones as m on i.milestone_id = m.id;
        // select
        // i.id, i.total, i.project_id, i.milestone_id,
        // i.datetime_generated, i.datetime_paid, i.datetime_void,
        // i.notes,
        // m.position,
        // p.about,
        // o.organization_name, o.organization_address, o.contact_name
        // from invoices as i
        // inner join milestones as m
        // on i.milestone_id = m.id
        // inner join projects as p
        // on i.project_id = p.id
        // inner join organizations as o
        // on p.organization_id = o.id
    }
}
```

```
public function fetchInvoiceByMilestoneIdFull($milestone_id){
    $invoice = DB::table('invoices as i')
        ->select(
            'i.id',
            'i.total',
            'i.project_id',
            'i.milestone_id',
            'i.datetime_generated',
            'i.datetime_paid',
            'i.datetime_void',
            'i.notes',
            'i.payment_id',
            'm.name',
            'm.position',
            'm.budget_percentage',
            'p.name as project_name',
            'p.about',
            'p.budget',
            'p.portal_domain',
            'o.organization_name',
            'o.organization_address',
            'o.contact_name',
            'o.contact_email',
            'o.contact_number'
        )
        ->join('milestones as m', 'm.id', '=', 'i.milestone_id')
        ->join('projects as p', 'i.project_id', '=', 'p.id')
        ->join('organizations as o', 'p.organization_id', '=', 'o.id')
        ->where('i.milestone_id', $milestone_id)
        ->first();

    return $invoice;
}

public function fetchInvoiceById($invoice_id){
    $invoice = DB::table('invoices')
        ->select('*')
        ->where('id', $invoice_id)
        ->first();

    return $invoice;
}

public function fetchInvoiceByMilestoneId($milestone_id){
    $invoice = DB::table('invoices')
        ->select('*')
        ->where('milestone_id', $milestone_id)
        ->first();

    return $invoice;
}
```

Figure 9.4.5.6.1. InvoicesController.php 1 of 2

```

public function createInvoiceOfMilestone(Request $req) {

    $exists = $this->fetchInvoiceByMilestoneId($req->milestone_id);
    if($exists){
        throw new \Exception('Invoice of this item exists already');
    }

    $milestone = DB::table('milestones')
        -> select('*')
        -> where('id', $req->milestone_id)
        -> first();

    $project = DB::table('projects')
        -> select('*')
        -> where('id', $milestone->project_id)
        -> first();

    $newId = DB::table('invoices')
        -> insertGetId([
            'project_id' => $milestone->project_id,
            'milestone_id' => $req->milestone_id,
            'total' => (($milestone->budget_percentage/100) * $project->budget),
            'notes' => null,
            'datetime_generated' => now(),
            'created_at' => now()
        ]);

    DB::table('notifications')
        -> insert([
            [
                'deliverable_id' => null,
                'project_id' => $milestone->project_id,
                'milestone_id' => $req->milestone_id,
                'type' => 'INVOICE',
                'status' => 'SENT',
                'description' => "Invoice has been created for $milestone->name",
                'extra' => null,
                'created_at' => now()
            ]
        ]);

    return $newId;
}

```

```

public function payInvoice($id, Request $req) {

    $invoice = DB::table('invoices')
        -> where('id', $id)
        -> first();

    DB::table('invoices')
        -> where('id', $id)
        -> update([
            'datetime_paid' => now(),
            'payment_id' => $req->payment_id,
            'payment_method' => $req->payment_method,
            'payment_client_secret' => $req->payment_client_secret
        ]);

    DB::table('notifications')
        -> insert([
            [
                'deliverable_id' => null,
                'project_id' => $invoice->project_id,
                'milestone_id' => $invoice->milestone_id,
                'type' => 'INVOICE',
                'status' => 'PAID',
                'description' => "Invoice has been paid for invoice #{$invoice->id}",
                'extra' => "Payment date: $invoice->datetime_paid",
                'created_at' => now()
            ]
        ]);

    return $invoice;
}

public function markInvoicePaid($id) {
    $invoice = DB::table('invoices')
        -> where('id', $id)
        -> first();

    DB::table('invoices')
        -> where('id', $id)
        -> update([
            'datetime_paid' => now()
        ]);

    DB::table('notifications')
        -> insert([
            [
                'deliverable_id' => null,
                'project_id' => $invoice->project_id,
                'milestone_id' => $invoice->milestone_id,
                'type' => 'INVOICE',
                'status' => 'PAID',
                'description' => "Invoice has been paid for invoice #{$invoice->id}",
                'extra' => "Payment date: $invoice->datetime_paid",
                'created_at' => now()
            ]
        ]);

    return $invoice;
}

```

Figure 9.4.5.6.1. InvoicesController.php 2 of 2

Notification Controller

Notification despite its all encompassing property has simple and elegant methods.

The method **addNotificationToProject** requires:

- notification **type** which can be (1) STATUS_UPDATE, (2) INVOICE, or (3) CHANGE
- notification **status** which will depend on the type
- **project_id**

nullable properties of addNotificationToProject are:

- **deliverable_id** if it is a notification related to a Deliverable
- **milestone_id** if it is a notification related to an Invoice or Milestone
- **description**
- **Extra** when the notification requires more information for context

The notification logic is applied to the code as a comment.

The notification can then use **markNotificationAsRead** once the Client has acknowledged.

```
class NotificationsController extends Controller
{
    // STATUS_UPDATE : {
    //     COMPLETE: '✅ ${attachmentType} has been completed',
    //     INCOMPLETE: '⚪ Hmm. A ${attachmentType} has been marked incomplete',
    // },
    // 'INVOICE':{
    //     SENT: '📄 Invoice has been sent',
    //     PAID: '💵 Invoice has been paid',
    // },
    // 'CHANGE':{
    //     'MADE': '✏️ ${attachmentType} has been changed',
    //     'CREATED': '💡 ${attachmentType} has been added',
    //     'DELETED': '🗑️ ${attachmentType} deleted',
    // }

    public function addNotificationToProject(Request $req, $project_id){
        DB::table('notifications')
            ->insert([
                [
                    'deliverable_id' => $req->deliverable_id,
                    'project_id' => $project_id,
                    'milestone_id' => $req->milestone_id,
                    'type' => $req->type,
                    'status' => $req->status,
                    'description' => $req->description,
                    'extra' => $req->extra,
                    'created_at' => now()
                ]
            ],
            true);
    }
}
```

Figure 9.4.5.7. NotificationsController.php

```
public function fetchNotificationById($id){
    $notification = DB::table('notifications')
        -> select('*')
        -> where('id', $id)
        -> first();
    return $notification;
}

public function fetchNotificationsByProject($project_id){
    $notifications = DB::table('notifications')
        -> select('*')
        -> where('project_id', $project_id)
        -> where('client_read_at', null)
        -> orderByDesc('created_at')
        -> get();
    return $notifications;
}

public function fetchNotificationsByProjectAsConsultant($project_id){
    $notifications = DB::table('notifications')
        -> select('*')
        -> where('project_id', $project_id)
        -> where('receiver_type', 1)
        -> where('client_read_at', null)
        -> orderByDesc('created_at')
        -> get();
    return $notifications;
}

public function markNotificationAsRead($id) {
    $notification = DB::table('notifications')
        -> where('id', $id)
        -> update([
            'client_read_at' => now()
        ]);
    return $notification;
}
```

Analytics Controller

The analytics controller is the most interesting of them all. The methods these includes are as follow

```
class AnalyticsController extends Controller
{
    public function fetchProgressPercentByProject($project_id){
        $totalDeliverables = DB::table('deliverables')
            ->select('*')
            ->where('project_id', $project_id)
            ->get();

        $complete = $totalDeliverables
            ->where('status', 'COMPLETE')
            ->count();

        $all = $totalDeliverables
            ->count();

        if($all==0){
            return 0;
        }
        return $complete / $all;
    }

    public function fetchProjectsAnalytics($user_id){
        $projectsController = new ProjectsController();
        $projects = $projectsController->fetchAllProjectsByConsultantUserId($user_id);

        $stats = array(
            'pending' => 0,
            'open' => 0,
            'due' => 0,
            'complete' => 0
        );

        $today = Carbon::now();
        $in30days = $today->addDays(30);

        foreach ($projects as $proj) {
            $progressPercent = $this->fetchProgressPercentByProject($proj->id);
            if($progressPercent == 0){
                $stats['pending'] += 1;
            }else if( $progressPercent == 1){
                $stats['complete'] += 1;
            }else{
                $stats['open'] += 1;
                if($proj->datetime_due != null && $proj->datetime_due < $in30days){
                    $stats['due'] += 1;
                }
            }
        }
    };

    return $stats;
}
```

(1) **fetchProgressPercentByProject** which calculates by number of Complete versus Incomplete Deliverables

(2) **fetchProjectAnalytics** which returns the number of Pending, Open, Due, and Complete projects of a specific Consultant.

Figure 9.4.5.8.1. Analytics Controller 1 of 3

- (3) **fetchMilestoneAnalytics** which returns (a) open milestones, (b) number of deliverables of an open milestone, (c) all deliverables overall complete or incomplete, (d) milestones due, (e) all deliverables completed.
- (4) **fetchInvoiceAnalytics** returns (a) all invoices of open projects, (b) paid invoices of open projects, (c) invoice paid last 30 days, (d) revenue last 30 days, (e) revenue the Consultant can collect

Figure 9.4.5.8.2. Analytics Controller 2 of 3

```
public function fetchMilestonesAnalytics($user_id){
    $projectsController = new ProjectsController();
    $projects = $projectsController->fetchAllProjectsByConsultantUserId($user_id);

    $stats = array(
        'open_milestones_id' => [],
        'deliverables_open_milestones_all' => 0,
        'deliverables_open_milestones_complete' => 0,
        'open_milestones_due' => 0,
        'deliverables_completed_all' => 0
    );

    $today = Carbon::now();
    $in30days = $today->addDays(30);
    foreach ($projects as $proj) {
        // deliverables
        $projDeliverables = DB::table('deliverables')
            ->select('*')
            ->where('project_id', $proj->id)
            ->get();

        $complete = $projDeliverables
            ->where('status', 'COMPLETE')
            ->count();
        $stats['deliverables_completed_all'] += $complete;

        $all = $projDeliverables
            ->count();
        $progress = $all > 0 ? $complete / $all : 0;

        if ($progress == 0){
            // if proj not started
        }else if ($progress == 1){
            // if proj complete
        }else{
            $ongoingMilestones = DB::table('deliverables')
                ->select('milestone_id')
                ->where('project_id', $proj->id)
                ->groupBy('milestone_id')
                ->whereIn('status', ['COMPLETE', 'INCOMPLETE'])
                ->havingRaw('COUNT(DISTINCT status) = 2') // Ensure both statuses are present
                ->pluck('milestone_id')
                ->toArray();

            $stats['open_milestones_id'] = array_merge($stats['open_milestones_id'], $ongoingMilestones);
            $stats['open_milestones_id'] = array_unique($stats['open_milestones_id']);
            foreach ($ongoingMilestones as $milestoneId){
                $milestoneDeliverable = DB::table('deliverables')
                    ->select('*')
                    ->where('milestone_id', $milestoneId)
                    ->get();
                $countCompleteDeliverable = $milestoneDeliverable
                    ->where('status', 'COMPLETE')
                    ->count();
                $stats['deliverables_open_milestones_all'] += $milestoneDeliverable->count();
                $stats['deliverables_open_milestones_complete'] += $countCompleteDeliverable;
            }

            $incompleteMilestones = $projDeliverables
                ->where('status', 'INCOMPLETE')
                ->pluck('milestone_id')
                ->toArray();

            $dueMilestones = DB::table('milestones')
                ->select('*')
                ->where('project_id', $proj->id)
                ->whereIn('id', $incompleteMilestones)
                ->where('datetime_due', '<', $in30days)
                ->count();
            $stats['open_milestones_due'] += $dueMilestones;
        }
    }

    $stats['open_milestones_id'] = array_values($stats['open_milestones_id']);
    return $stats;
}
```

```
public function fetchInvoicesAnalytics($user_id){
    // get open projects

    $projectsController = new ProjectsController();
    $projects = $projectsController->fetchAllProjectsByConsultantUser($user_id);

    $stats = array(
        'open_project_invoice_all_ids' => [],
        'open_project_invoice_paid_ids' => [],
        'invoices_paid_last_30_days_ids' => [],
        'revenue_received_last_30_days' => 0,
        'revenue_collectible' => 0
    );

    $today = Carbon::now();
    // $in30days = $today->addDays(30);
    $last30days = Carbon::now()->subDays(30);
    $startOfMonth = Carbon::now()->startOfMonth();
    $endOfMonth = Carbon::now()->endOfMonth();

    $stats['today'] = $today;
    $stats['start_of_month'] = $startOfMonth;

    // dd($startOfMonth->toDateTimeString(),
    // $endOfMonth->toDateTimeString());

    foreach ($projects as $proj) {
        $progressPercent = $this->fetchProgressPercentByProject($proj->id);
        $allProjInvoices = DB::table('invoices')
            ->select('*')
            ->where('project_id', $proj->id)
            ->get();

        foreach ($allProjInvoices as $inv){
            if($inv->datetime_paid != null){
                // $dateTimePaid = Carbon::parse($inv -> datetime_paid);
                if(
                    $inv->datetime_paid > $startOfMonth
                ){
                    $stats['invoices_paid_last_30_days_ids'][] = $inv->id;
                    $stats['revenue_received_last_30_days'] = $inv->total;
                }
            }else{
                $stats['revenue_collectible'] += $inv->total;
            }
        }

        if($progressPercent == 0){
            // $stats['pending'] += 1;
        }else if ($progressPercent == 1){
            // $stats['complete'] += 1;
        }else{
            // OPEN PROJECTS
            foreach ($allProjInvoices as $inv){
                $stats['open_project_invoice_all_ids'][] = $inv->id;

                if($inv->datetime_paid != null ||
                    $inv->payment_client_secret != null){
                    $stats['open_project_invoice_paid_ids'][] = $inv->id;
                }
            }
        }
    }
}
```



```

public function matchMilestonesToInvoice($user_id){
    $completeMilestones = [];
    $stats = array(
        'complete_milestones_ids' => [],
        'milestones_with_invoice_ids' => [],
        'milestones_missing_invoice_ids' => [],
    );

    $projectsController = new ProjectsController();
    $projects = $projectsController->fetchAllProjectsByConsultantUserId(
        $user_id);

    foreach ($projects as $proj){
        $allProjMilestones = $totalDeliverables = DB::table(
            'milestones')
            ->select('*')
            ->where('project_id', $proj->id)
            ->get();

        foreach ($allProjMilestones as $ms){
            $totalDeliverables = DB::table('deliverables')
                ->select('id')
                ->where('milestone_id', $ms->id)
                ->count();

            $completeDeliverables = DB::table('deliverables')
                ->select('id')
                ->where('milestone_id', $ms->id)
                ->where('status', 'COMPLETE')
                ->count();

            if($completeDeliverables/$totalDeliverables){
                $stats['complete_milestones_ids'][] = $ms->id;
            }
        }
    }

    foreach($stats['complete_milestones_ids'] as $msId){
        $hasInvoice = DB::table('invoices')
            ->select('*')
            ->where('milestone_id', $msId)
            ->count();

        if($hasInvoice > 0){
            $stats['milestones_with_invoice_ids'][] = $msId;
        }else{
            $stats['milestones_missing_invoice_ids'][] = $msId;
        }
    }

    return $stats;
}

```

(5) **matchMilestonesToInvoice** returns the invoice ids of (a) milestones with matching invoice and (b) milestones without invoices

(6) **fetchProgressPercentByMilestone** returns the completion rate of milestone depending on the complete versus incomplete deliverables

Figure 9.4.5.8.3.. Analytics Controller 3 of 3

```

public function fetchProgressPercentByMilestone($milestone_id){
    $totalDeliverables = DB::table('deliverables')
        ->select('*')
        ->where('milestone_id', $milestone_id)
        ->get();

    $complete = $totalDeliverables
        ->where('status', 'COMPLETE')
        ->count();

    $all = $totalDeliverables
        ->count();

    if($all==0){
        return 0;
    }

    $milestone = DB::table('milestones')
        ->select('*')
        ->where('id', $milestone_id)
        ->first();

    return array(
        'completion_rate'=> $complete / $all,
        'milestone' => $milestone
    );
}

```

5. Frontend Implementation Codebase

Our frontend directory is found in `./scopecliq-ui`

5.1. Routes and Views

App.js

The URL routes are controlled by App.js. You can find here the main views and the url formats that lead to them.

```
const App = () => {
  const dispatch = useDispatch();
  const snackbar = useSelector(storeSnackbar);

  useEffect(() => {
    if (snackbar.show) {
      setTimeout(() => {
        dispatch(resetSnackbarMessage())
      }, 5000)
    }
  }, [snackbar])

  return (
    <div className="scopecliq">
      <Routes>
        <Route path="/" element={<Home/>} />
        <Route path="/dashboard/:projectId" element={<ConsultantDashboard/>} />
        <Route path="/dashboard" element={<DashboardHomeLayout/>} />
        <Route path="/portal/:domain" element={<ClientPortal/>} />
        <Route path="/invoice/:paramMilestoneId" element={<Invoice/>} />
        <Route path="/portal/:domain/invoices" element={<InvoiceListLogin/>} />
        <Route path="/dashboard/:projectId/invoices" element={<InvoiceList/>} />
      </Routes>
      <Snackbar open={snackbar.show}>
        <div className={`
          sq-outer-shadow border-sq-light px-3 py-2 rounded text-color-sq-white
          font-size-14
          ${!snackbar.status && ' bg-sq-gold-mid '}
          ${snackbar.status == 'error' && ' bg-sq-tomato-light '}
        `>
          {snackbar.message}
        </div>
      </Snackbar>
    </div>
  );
}

export default App;
```

Figure 9.5.1.1. App.js holds the routes and the Snackbar Functionality that is applied all over the app

Our main view components are

- (1) **Home**
- (2) **ConsultantDashboard.js** – see [2.1. Consultant Dashboard > Consultant Project Page](#)
- (3) **DashboardHomeLayout.js** – see [2.2 Consultant Dashboard > Consultant Homepage](#)
- (4) **ClientPortal.js** – see [1. Client Portal](#)
- (5) **Invoice.js** – see [Global Invoice Link for a Single Invoice Item and Accessibility](#)
- (6) **InvoiceList.js** and **InvoiceListLogin.js**– see [Invoice Lists Entry Points and Accessibility](#)

Index.js and config.js

There is a file higher than App.js. **Index.js** holds the react-redux controller as seen from using the **<Provider>** tag.

```
import './index.css';
import './config.js';

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <Provider store={store}>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </Provider>
);
```

Index.js also holds **index.css** (which applies all the styling) and **config.js** (which defines the backend api route we will be using to connect to the database)

Figure 9.5.1.2. Index.js

```
module.exports = global.config = {
  API: "http://localhost:8000/api"
  // API: "https://scopecliq.com/api"
}
```

Figure 9.5.1.3. config.js defines the api route depending on if we are running on local or production.

Styling

We are using **SASS-SCSS** to style the whole app.

style.scss

style.scss imports all the required styling.

variable.scss has all our colors

typography.scss handles the headings, paragraphs, links, and other special labels

buttons-and-forms.scss handles buttons, inputs, text areas

mains.scss handles the main layout structure. This includes sidebars, navigation, and main content.

milestone.scss and **projects.scss** handles the Project Blueprint and sub component styling

notification.scss styles the notification depending on status and state

invoices.scss handles invoice and invoice list.

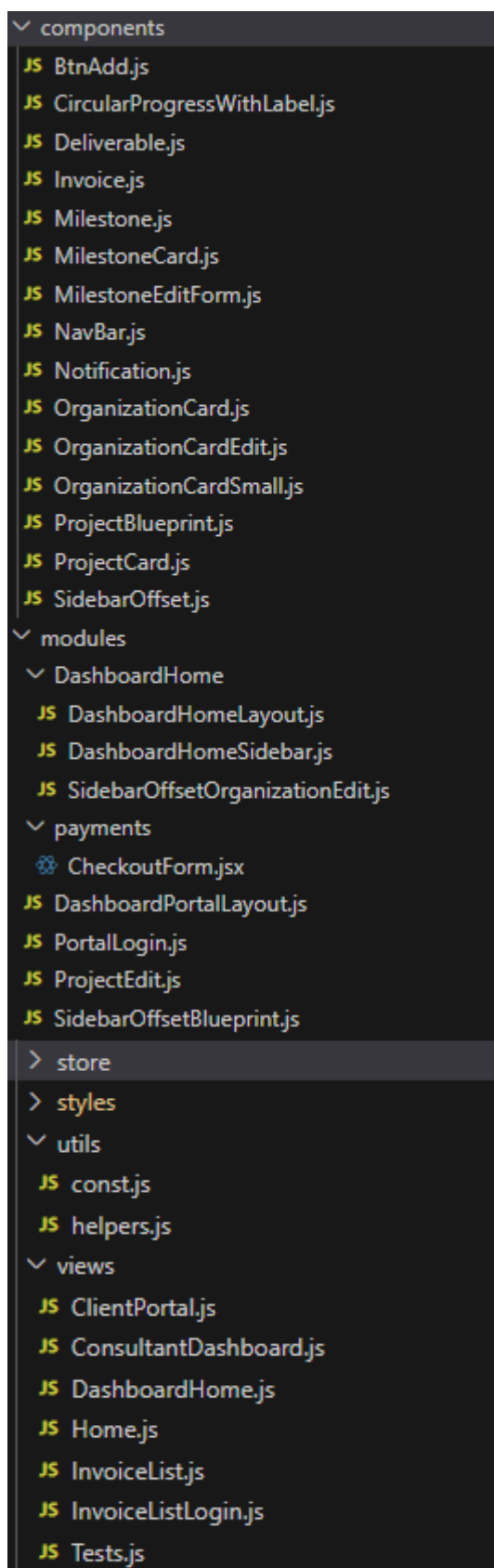
Figure 9.5.1.3. style.scss

```
@import url('https://fonts.googleapis.com/css2?family=DM+Sans
wght@0,100;0,200;0,300;0,400;0,500;0,600;0,700;0,900;0,1000;1
200;1,300;1,400;1,500;1,600;1,700;1,800;1,1000&display=swap');

@import './utils/variables.scss';
@import './utils/borders-and-bgs.scss';
@import './utils/typography.scss';
@import './utils/effects.scss';
@import './utils/buttons-and-forms.scss';

@import './modules/main.scss';
@import './modules/milestones.scss';
@import './modules/notifications.scss';
@import './modules/projects.scss';
@import './modules/invoices.scss';

*{
  font-family: 'DM Sans', sans-serif;
}
body{
  line-height: 1.4;
  color: $sq-med;
  background-color:$sq-lightest;
  transition: all 0.4s ease;
  &::-webkit-scrollbar {
    display: none;
    transition: 1s all ease;
    opacity: 0;
    background: transparent; /* Optional: just make scro
invisible */
    height: 0.8rem;
    width: 0.5rem;
    border: 1px solid $sq-lighter;
  }
  &::-webkit-scrollbar-thumb {
    border-top: 1px solid #ddd;
    background-color: $sq-lav-light;
  }
}
```



5.2. Components and Modules

We have an extensive list of Components and Modules. For the frontend section we will just list some of the notable code features.

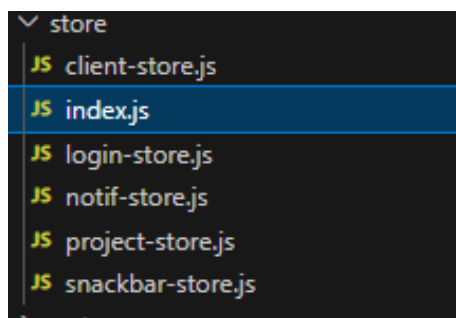
The **component** directory holds the components that are being reused multiple times.

The **module** directory hold components that act as a group and work together

The **views** directory holds components that act as holder for the routes as you can see from [App.js](#)

Figure 9.5.2. Frontend directory of Components and Modules

5.3. React-redux or the Store



We are able to distribute the data seamlessly through different components using react-redux.

You can find our react-redux components from ./store

Figure 9.5.3.1. The ./store directory holding the reducers

```
import { configureStore } from '@reduxjs/toolkit'
import userReducer from './client-store'
import projectReducer from './project-store'
import loginReducer from './login-store'
import snackbarReducer from './snackbar-store'
import notifReducer from './notif-store'

export const store = configureStore({
  reducer: {
    project: projectReducer,
    login: loginReducer,
    user: userReducer,
    snackbar: snackbarReducer,
    notif: notifReducer
  }
})
```

Figure 9.5.3.2. ./store/index.js holding the reducers and allowing them transcend all over the website with the <Provider> from [Index.js](#)

```
import { createSlice } from '@reduxjs/toolkit'

const notifSlice = createSlice({
  name: 'notif',
  initialState: {
    control: 0
  },
  reducers: {
    updateNotif(state) {
      state.control += 1
    },
  }
})

export const notifControl = (state) => state.notif.control
export const { updateNotif } = notifSlice.actions
export default notifSlice.reducer
```

notif-store.js is an example of a reducer. This javascript file allows the components nested in **DashboardPortalLayout > Milestone > Deliverables** to automatically **update** the component **DashboardPortalLayout > SidebarOffsetBlueprint > Notifications** instead of having to refresh every time there is a new activity

5.4. Notable Features and Snippets

```
const snackbarSlice = createSlice({
  name: 'snackbar',
  initialState: {
    snackbarActive: {
      show: false,
      status: null,
      message: ""
    }
  },
  reducers: {
    showSnackbarMessage(state, action){
      console.log({action, state})
      state.snackbarActive = {
        show: true,
        status: action.payload.status,
        message: action.payload.message
      };
    },
    resetSnackbarMessage(state){
      state.snackbarActive = {
        show: false,
        status: null,
        message: ""
      }
    }
  }
})
export const storeSnackbar = (state) => state.snackbar.snackbarActive
export const { showSnackbarMessage, resetSnackbarMessage } = snackbarSlice.actions
export default snackbarSlice.reducer
```

Snackbars

Snackbars are essential in showing successes and errors of the actions.

We were able to implement a Snackbar System throughout the website by applying it in [App.js](#) and dispatching the reducers all through-out.

A good example of Snackbar use is in **error-handling**.

We wrap a request in a try- catch, and if errors arise, we let it be known by displaying a snackbar.

Figure 9.5.4.1. (left)

snackbar-store.js

Figure 9.5.4.2. (right)

Registration function found in **Home.js** using snackbars for when the registration succeeds or fails

Figure 9.5.4.3. (bottom)

Snackbar use from error-handling in action

```
const register = async() => {
  if(!isRegistrationValid({
    email: modelEmail,
    password: modelPassword
  })){ return }
  if(!isOrganizationValid()){ return }
  console.log('attempting registration...')
  try {
    const response = await axios.post(api+ '/user/register', {
      name: organization.contact_name,
      email: modelEmail,
      password: modelPassword
    }, {
      headers: {
        "Content-Type": "application/json",
      }
    });
    if(response?.data?.status==='success'){
      const userId = response.data.user_id
      dispatch(setUserId(userId))
      dispatch(showSnackbarMessage({
        message: "Account created"
      }))
      await createOrganization(userId)
    }
  } catch (error) {
    console.log(error)
    dispatch(showSnackbarMessage({
      status: "error",
      message: error.response.data.message | error.message
    }))
  }
}
```

```
import { createSlice } from '@reduxjs/toolkit'

const loginSlice = createSlice({
  name: 'login',
  initialState: {
    isLoggedIn: false,
    userId: null,
    user: null,
    userOrg: null,
  },
  reducers: {
    toggleLogin(state){
      state.isLoggedIn = !state.isLoggedIn
    },
    setLogin(state, action){
      state.isLoggedIn = action.payload
    },
    setUser(state, action){
      state.user = action.payload
    },
    setUserOrg(state, action){
      state.userOrg = action.payload
    },
    setUserId(state, action){
      if(action.payload){
        localStorage.setItem("sq_user_id", action.payload)
      }else{
        localStorage.removeItem("sq_user_id")
      }
      state.userId = action.payload
    }
  }
})

export const isLoggedIn = (state) => state.login.isLoggedIn
export const currentUser = (state) => state.login.user
export const currentUserOrg = (state) => state.login.userOrg
export const currentUserOrg = (state) => state.login.userOrg

export const currentUserOrg = (state) => state.login.userOrg
export const currentUserOrg = (state) => state.login.userOrg

export const { toggleLogin, setLogin, setUser, setUserOrg } = loginSlice.actions
export default loginSlice.reducer
```

Use of the browser's local storage

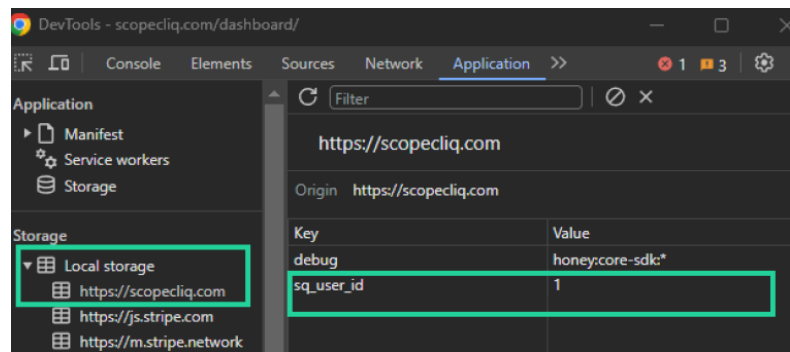
To save the state of whether we are logged in we use the browser's **local storage**.

Everytime a Consultant successfully logs in the user id saved in your local storage under `sq_user_id`.

This ensures that the browser knows who we are. This local storage key is removed on logout.

Figure 9.5.4.4. (left) `localStorage` used in `loginSlice` reducer a

Figure 9.5.4.5. (bottom) `sq_user_id` assigned the value of 1 as seen when browser is inspected



React Material UI Design Library

We used **React's Material UI library** instead of building components from scratch. Examples of the usage are the tooltips, the circular progress notification, and modals.

```
<h4>Milestones and Deliverables</h4>
<div className='d-flex align-items-stretch mb-2'>
  <div className='sq-stat-card d-flex flex-column justify-content-center align-items-center p-2 bg-sq-lav-light rounded w-50 me-2 text-center'>
    <div className='sq-stat-card-info-holder'>
      <Tooltip title="
        Once you have completed one Deliverable in a milestone, it is now considered an open milestone.
      ">
        <div className='sq-btn-icon'>
          <i className='fa-solid fa-question' />
        </div>
      </Tooltip>
    </div>
    <span className='h2 text-color-sq-lav-mid mb-0'>
      {milestoneStats.open_milestones_id.length}
    </span>
    <div className='p mt-0'>
      Ongoing milestones
    </div>
  </div>
  <div>
    <span>2</span>
    <div>
      3 of 8 Deliverables
    </div>
    <div>
      Once you have completed one Deliverable in a milestone, it is now considered an open milestone.
    </div>
  </div>
  <div>
    1
    <div>
      Due milestone in the next 30 days
    </div>
  </div>
  <div>
    24 total completed deliverables
  </div>
</div>
```

Figure 9.5.4.6. (above) Use of `<Tooltip>` component from Material UI

Figure 9.5.4.7. (right) Customization of `CircularProgress` into `CircularProgressWithLabel` component

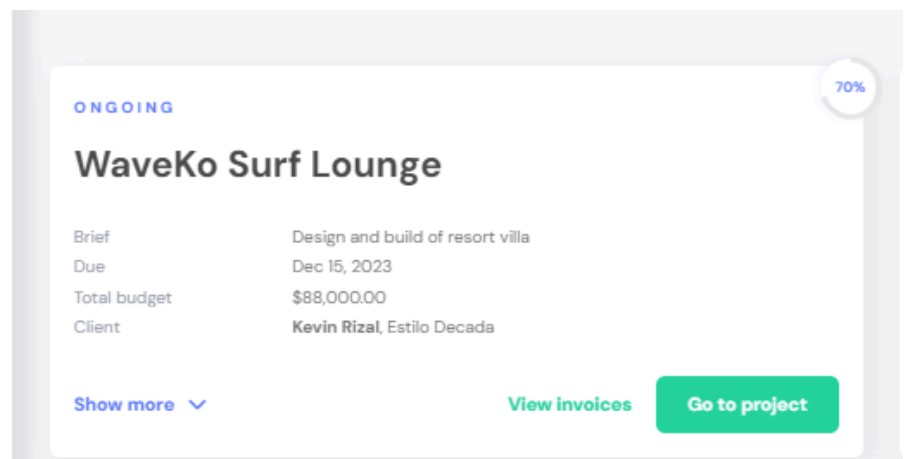
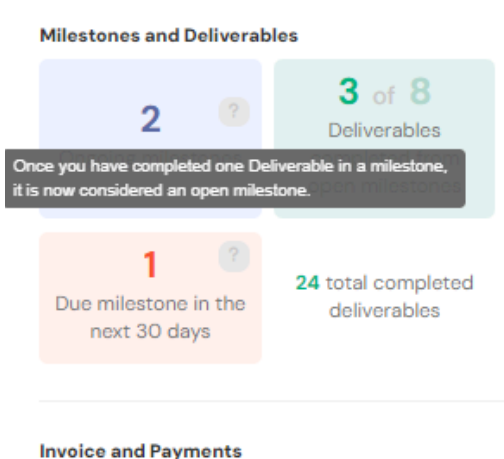
Figure 9.5.4.8. (below) mentions components used on Consultant Homepage

```
import * as React from 'react';
import CircularProgress, {
  CircularProgressProps,
} from '@mui/material/CircularProgress';
import Box from '@mui/material/Box';

function CircularProgressWithLabel(
  {value, color="#E7E7E7"}
) {
  const circularProgressStyle = {
    strokeWidth: '12px',
    color: color,
  };

  return (
    <Box sx={{ position: 'relative', display: 'inline-flex'}}>
      <CircularProgress variant="determinate" value={value} style={circularProgressStyle} />
      <Box
        sx={{
          top: 0,
          left: 0,
          bottom: 0,
          right: 0,
          position: 'absolute',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <div className='sq-label'>
          {`${Math.round(value)}%`}
        </div>
      </Box>
    </Box>
  );
}

export default function CircularWithValueLabel({value}) {
  const [progress, setProgress] = React.useState(10);
  return <CircularProgressWithLabel value={value} />;
}
```



Stripe

Invoice.js

See [Payments and Stripe Integration](#)

```
const fetchPaymentIntent = async () =>{
  if(!invoice?.total || invoice?.total <= 0) return
  if(invoice?.datetime_paid) return
  try{
    const paymentIntent = await stripe.paymentIntents.create({
      amount: parseInt(invoice.total),
      currency: "cad",
      // In the latest version of the API, specifying the
      `automatic_payment_methods` parameter is optional because
      Stripe enables its functionality by default.
      automatic_payment_methods: {
        enabled: true,
      },
    });
    console.log({paymentIntent})
    if(paymentIntent){
      setPaymentIntent(paymentIntent)
    }
    if(paymentIntent.client_secret){
      setClientSecret(paymentIntent.client_secret)
      setOptions({
        clientSecret: paymentIntent.client_secret
      })
    }
  }catch(e){
    console.log(e)
  }
}
```

Once customer initiates payment, we run the **fetchPaymentIntent** method as accessing Stripe's server.

If payment is successful a `client_secret` is returned which we save on our own database as we match the payment as complete.

Figure 9.5.4.9. fetchPaymentIntent using imported Stripe class

Figure 9.5.4.10. payInvoice method transforms our data to match the intent.

```
const payInvoice = async (paymentIntent) =>{
  if(!paymentIntent || paymentIntent.status !== "succeeded") return
  const payload = {
    id: invoice.id,
    payment_id: paymentIntent.id,
    payment_method: paymentIntent.payment_method,
    payment_client_secret: paymentIntent.client_secret
  }
  try{
    const res = await axios.post(`${api}/invoices/pay/${invoice.id}`, payload, {
      headers: {
        "Content-Type": "application/json",
      },
    })
  }catch(e){
    console.log(e)
  }
  dispatch(showSnackBarMessage({
    message: "Payment successful"
  }))
  getInvoiceDetails()
}
}
```

References

1. <https://www.enterpriseappstoday.com/stats/freelancing-statistics.html>
Enterprise Apps Today – Berry Elad
2. <https://www.forbes.com/sites/elainepofeldt/2017/10/17/are-we-ready-for-a-workforce-that-is-50-freelance>
Are We Ready For A Workforce That is 50% Freelance? – Elaine Pofeldt
3. <https://www.dubsado.com/>
4. <https://weareindy.com/>
5. <https://www.honeybook.com/>
6. <https://www.atlassian.com/software/jira>
7. <https://asana.com/>
8. <https://trello.com>
9. <https://forms.gle/DeoWtVRndo78yv659>
ScopeCliq – Client / Independent Contractor Questionnaire
10. <https://world.siteground.com/>
11. <https://getbootstrap.com/>
12. <https://fontawesome.com/icons>
13. <https://mui.com/material-ui/>